An Interpretable Dictionary Learning Approach for Abnormality Detection and Segmentation in Chest CT

An undergraduate project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

Degree of the Bachelor of the Science of Engineering Honours

by

K.R.C.S. Kekulawala - EG/2021/4609
B.G.P.L. Thejana - EG/2021/4828
B.G.P.U. Thilakshana - EG/2021/4831
M.D.Y.B. Kularathne - EG/2021/4620

.....	.....
Dr. Kaveen Liyanage	Ms. Sithara Mahagama
(Supervisor)	(Co-Supervisor)

Abstract

Automated abnormality detection from 3D chest Computed Tomography requires clinically interpretable methods to support radiological decision-making. Current State-of-the-Art (SotA) deep learning models often rely on post-hoc explainability techniques that provide approximate localizations and lack direct traceability to the models decisions. Furthermore, SotA models rarely support simultaneous multi-class classification and grounded localization, leading to a fragmented approach to diagnosis. This also hinders the ability to benchmark models across multiple dimensions of performance, including classification accuracy, segmentation quality, and explainability. This study aims to address these challenges through two contributions: First, we introduce an inherently interpretable dictionary learning approach in which detected abnormalities can be back-projected to the original voxel space, allowing diagnostic decisions to be traced back to relevant anatomical structures. Second, we develop a unified evaluation framework to benchmark SotA models across three axes: (i) classification accuracy, (ii) segmentation quality, and (iii) explainability, using standardized metrics and a publically available 3D chest CT dataset. Our method is benchmarked against SotA models using this framework, demonstrating competitive classification and segmentation performance while providing improved interpretability compared to deep learning baselines.

Acknowledgments

We would like to express our sincere gratitude to Dr. Kaveen Liyanage (Supervisor), Mrs. Sithara Mahagama (CoSupervisor), and Dr. Subhodini Indrapala (Consultant Radiologist at Teaching Hospital, Karapitiya), for their invaluable guidance, support, and constructive feedback throughout the course of this project. Their expertise, mentorship, and encouragement have been immensely helpful in shaping the direction and quality of this research. We are grateful to the Department of Electrical and Information Engineering for providing access to high-performance computing resources and research facilities that were essential for conducting this project. The department's commitment to fostering an environment conducive to research and innovation is greatly appreciated. We would like to acknowledge the creators and maintainers of the publicly available datasets used in this research, particularly the CT-RATE and RexGroundingCT datasets and other medical imaging resources that have made this work possible. These datasets have been crucial for developing and validating the proposed methods. We extend our thanks to the developers and maintainers of the open-source frameworks and libraries used in this project, which significantly contributed in the implementation of this research.

Preface

This thesis is submitted.....

Contents

List of Figures

List of Tables

Acronyms

CXR	-	Chest X-Ray
CT	-	Computed Tomography
CAD	-	Computer-Aided Diagnosis
CNN	-	Convolutional Neural Network
ViT	-	Vision Transformer
XAI	-	Explainable Artificial Intelligence
GGO	-	Ground-Glass Opacity
GGN	-	Ground-Glass Nodule
SotA	-	State of the Art
PACS	-	Picture Archiving and Communication Systems
LIME	-	Local Interpretable Model-Agnostic Explanations
SHAP	-	SHapley Additive exPlanations
DeepLIFT	-	Deep Learning Important FeaTures
CAM	-	Class Activation Mapping
Grad-CAM	-	Gradient-weighted CAM
HU	-	Hounsfield Unit

Chapter 1

Introduction

Developing a unified pipeline for multi-class abnormality classification and grounded localization addresses two of the most critical imperatives in clinical AI adoption: diagnostic accuracy and explainability. Specifically, utilizing dense pixel-level segmentation as the mechanism for localization forces the architecture to provide voxel-level visual evidence for its diagnostic claims. This ensures that the model is accountable and its claims are supported by clinical evidence.

1.1 Background

Lung disease remains a leading cause of global morbidity and mortality [1]. Research shows that early detection of pulmonary diseases significantly improves patient outcomes. For this, radiological imaging tools such as Chest X-Ray (CXR) and Computed Tomography (CT), plays an important role. However, the increasing volume of such imaging data increases radiologists' workload, leading to delays and potential errors in diagnosis. This has motivated the development of automated Computer-Aided Diagnosis (CAD) systems, which use computational algorithms and Artificial Intelligence (AI) to assist in tasks such as abnormality detection, classification, and segmentation [2].

Our research mainly focuses on the development of a CAD system using 3 Dimensional (3D) CT data. While CXR is commonly used as the first imaging test due to its low cost and low radiation exposure, studies show that their sensitivity to abnormailities is limited [3]. Comparatively, CT scans produce high resolution cross-sectional slices that helps detect smaller, early-stage, and subtle lung pathologies that are frequently missed by standard CXR.

However, CT analysis is complex and time-consuming, requiring radiologists to review hundreds of slices per scan, which are often inconsistent due to artifacts, anatomical variations and different scanners/protocols. Furthermore, different anatomical structures can share identical grayscale values. Telling them apart requires expert clinical judgment which is often subjective. High sensitivity can lead to detection of clinically insignificant abnormalities, while visual fatigue can lead to missed findings [4].

To address these challenges, recent research and clinical deployments of AI-based CAD systems are fundamentally changing how radiologists approach CT analysis. AI models such as 3D Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) have shown to reduce interpretation time, and improve lesion detection sensitivity compared to traditional manual methods. Many of these high perform-

ing models are Deep Learning based, which are criticized for their black box nature, where their internal reasoning is difficult to interpret. This lack of interpretability is a significant barrier for clinical adoption, as radiologists require evidence to support a model's diagnostic claims.

This has motivated the development of explainable AI (XAI) methods such as Gradient-weighted Class Activation Mapping (Grad-CAM) that helps to visualize exactly which parts of an image a model focuses on when making a specific prediction. Such XAI methods are highly valuable for building clinical trust in deep learning models. However, they often lack the fine-grained, pixel-level precision required to identify tiny lesions, sometimes highlighting incorrect or confounding features [5].

Recent advances in sparse representation models such as Sparse Autoencoders and Dictionary Learning have shown to provide inherent interpretability in medical imaging. This stems from their ability to reconstruct complex, high-dimensional data as a linear combination of "atoms" or features. This allows the use of efficient algorithms to easily map the output sparse domain back to the original input feature space, providing a linear relationship between the model's decision and anatomical structures. Our work builds on these advances by applying dictionary learning to 3D Chest CT data for multi-class abnormality classification and grounded localization.

1.2 Problem Statement

The clinical utility of Deep Learning based approaches is hindered by a lack of interpretability. While tools like Grad-CAM provide post-hoc explanations to model decisions, these are often insufficient for medical professionals who require a granular understanding of the model's decision making process. In high stakes environments, a model that produces a diagnosis without transparency in its reasoning is difficult to integrate into clinical workflows, as it lacks the accountability necessary for legal and ethical compliance.

1.3 Objectives and Scope

The primary goal of this study is to address the limitations of existing AI-based approaches for abnormality classification and localization in chest CTs, by developing an inherently interpretable dictionary learning framework. This approach aims to enable transparent, anatomically grounded decision making, where each classification outcome can be traced back to specific pathological patterns. This aligns with clinical reasoning and facilitates validation and adoption in radiological workflows. The specific objectives are as follows:

1.3.1 Objectives

- Implement a Dictionary Learning based CAD system using a publicly available annotated 3D chest CT dataset.
- Map dictionary atoms back to anatomical features and visualize their spatial contribution maps.

- Develop a unified evaluation framework for assessing the performance and interpretability of the proposed approach.
- Benchmark model performance against SotA classification and segmentation models on the same dataset.

1.3.2 Scope

The scope of this study focuses on the development and evaluation of a dictionary learning based approach for classification and grounded localization of lung abnormalities using volumetric 3D CT data, with the following limitations:

- This study is limited to 2 thoracic abnormalities: Pulmonary Nodules, and Ground-Glass Opacities, based on expert radiologist recommendation and dataset availbility.
- This study will utilize a single publicly available dataset for training and evaluation, which may limit generalizability to other populations and scanner types.
- The proposed approach will be limited only to interpretable methods, and will not explore approaches that provide better performance for less explainability.

Chapter 2

Literature Review

2.1 Clinical Background

Chronic respiratory diseases account for more than 4 million deaths globally per year [1]. Research shows that early detection of pulmonary diseases, particularly lung cancer and Chronic Obstructive Pulmonary Disease (COPD) dramatically lowers mortality rates. Two landmark studies, the National Lung Screening Trial (NLST) and the Nederlands-Leuvens Longkanker Screenings Onderzoek (NELSON) trial, have shown that low-dose CT (LDCT) screening reduces lung cancer mortality by 20% to 33% compared to CXR [6, 7].

2.1.1 Computed Tomography (CT)

CTs generate 3D structural information by acquiring 2D radiographic projections of an object from many angles and computationally reconstructing them into a stack of cross-sectional slices via algorithms such as filtered backprojection [8]. Figure 2.1 shows the Backprojection reconstruction process for a single CT slice.

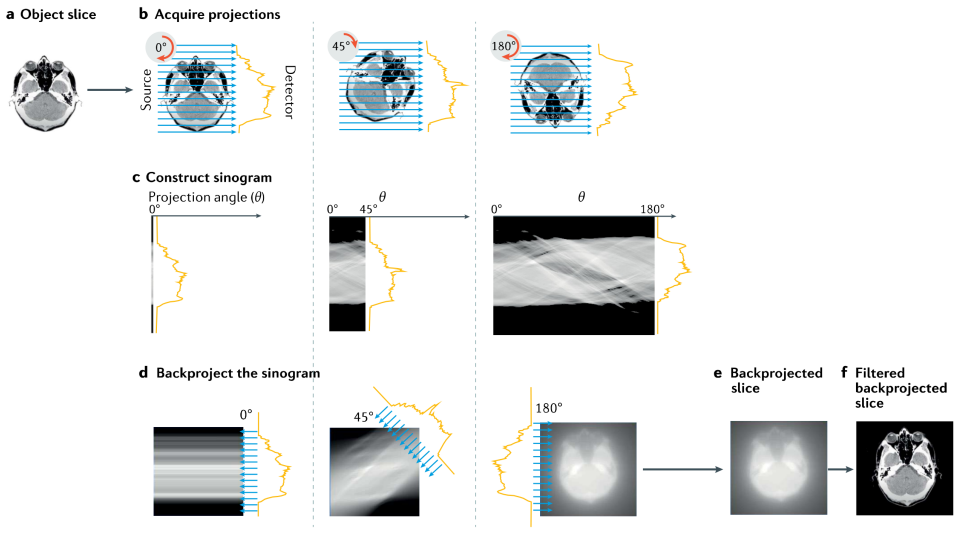

Figure 2.1: Backprojection reconstruction method for a single slice. (a - Original object slice. b - Set of projections collected at different angles. c - Sinogram resulting from many projections. d - Process of backprojecting the sinogram. e - Final backprojected imaqe. f - Equivalent filtered backprojection image).

Compared with other imaging modalities, CT's central advantage is its non-destructive, quantitative access to internal 3D structure. CT can visualize and quantify internal features throughout an intact volume, enabling longitudinal or repeated imaging of the same specimen over time. Attenuation-contrast CT, the dominant mode in clinical practice, works well for high-atomic-number, strongly attenuating structures such as bone or calcified tissue and, with contrast agents, vascular or soft-tissue structures. However, it provides relatively poor contrast between materials of similar composition and density (e.g., different soft tissues), a limitation that phase-contrast and other advanced modalities partially address, though these are less widely deployed clinically due to hardware and dose constraints.

Interpreting 3D chest CT specifically is complicated by several factors described in the imaging literature. Reconstructions are inherently affected by noise and artifacts, motion from breathing or cardiac movement, ring artifacts, and the partial volume effect, where a single voxel spans more than one tissue type, producing intermediate intensities that blur boundaries between structures. These effects broaden and overlap the intensity histograms of different tissues, making simple threshold-based segmentation unreliable and often necessitating boundary-based, region-growing, or learning-based segmentation methods instead. These challenges motivate the shift toward automated, learning-based segmentation approaches capable of handling such structural diversity without extensive manual tuning per dataset.

2.1.2 Pulmonary Abnormalities

Our study focuses on the detection and segmentation of five common pulmonary abnormalities:

1. Ground-Glass Opacity (GGO)
2. Pulmonary nodules
3. Atelectasis
4. Consolidation
5. Bronchiectasis

of which the first two were taken as the primary targets for the initial phase of this study.

A nodule is a discrete, roughly rounded area of increased opacity within the lung parenchyma, generally classified as either a solid nodule or a ground-glass nodule (GGN) based on its internal composition. A GGO refers specifically to a hazy area of increased lung attenuation on CT that does not obscure the underlying bronchial or vascular structures passing through it. GGNs are further subdivided into pure GGNs, which contain no solid component, and mixed (subsolid) GGNs, which contain a solid portion alongside the ground-glass components.

These findings are clinically important because GGNs are frequently associated with early lung cancer and its preinvasive precursors. Notably, GGNs are not uniformly benign or malignant risk indicators equivalent to solid nodules. GGNs are, in fact, more likely to be malignant than solid nodules, and the malignant rate of mixed GGNs is higher than that of pure GGNs.

Early detection of these abnormalities is essential because lung cancer prognosis is highly stage-dependent, and most patients are still diagnosed too late for optimal outcomes. Despite improvements in treatment, the overall five-year survival rate for lung cancer remains only about 10-20% in most countries, largely because most patients are diagnosed at a late stage.

Despite their importance, GGOs and pulmonary nodules remain difficult to detect and characterize reliably. Nodules may be small relative to other anatomical structures in a chest CT, and GGO may be low-contrast against surrounding tissue. These challenges make manual interpretation difficult and motivate the shift toward automated, learning-based segmentation approaches.

2.2 Computer-Aided Diagnosis (CAD) Systems

CAD systems are computer based tools that help medical personnel in analysis of imaging studies and help in making better diagnostic decisions. CAD does not intend to replace doctors and instead, acts as a second opinion by identifying suspicious areas in medical images and providing useful information to support diagnosis. The final diagnosis is always made by the doctor[9].

2.2.1 Evolution of CAD Systems

The modern CAD framework was defined in the 1980s when the University of Chicago shifted the objective from automated diganosis to acting as a second reader [9]. Early iterations relied on manual image processing using edge detection, filtering, and feature extraction to locate nodules in chest X-rays or microcalcifications in mammograms. Chan et al. provided a pivotal study showing that radiologists using CAD identified more clustered microcalcifications than those working alone, despite a high rate of false positives [10].

Over the 1990s and 2000s, CAD spread to areas such as lung cancer detection on chest X-rays and CT scans, detection of vertebral fractures linked to osteoporosis, brain aneurysm detection using MR angiography, and tracking interval changes in bone scans [9]. A prospective study on mammography, covering tens of thousands of patients, showed a steady rise in cancer detection rates using CAD [11]. This included finding small invasive tumors earlier. Sensitivity for microcalcification clusters rose from roughly 87% in the early 1990s to about 98% by the mid-2000s, while false positives dropped [12]. This "classical CAD" era relied on statistical pattern recognition and manually engineered features. Deep learning has since become the dominant approach in CAD.

The purpose of CAD has also shifted from early tools focused on detection by flagging suspicious zones, to later versions, which added differential diagnosis to estimate if a nodule was malignant or benign. Some systems even retrieve similar past cases from hospital Picture Archiving and Communication Systems (PACS) for comparison. Observer studies indicate that likelihood-of-malignancy scores improve radiologist decisions in subtle cases. Doctors still rely on their own judgment for obvious cases while CAD provides the most utility in borderline situations [13].

Future CAD development will likely move away from stand-alone tools. Instead, these will be integrated packages within PACS, allowing a single scan to be screened for multiple conditions simultaneously [14]. Deep learning has sped up this process.

A single trained model can often be adapted for various abnormalities with far less manual effort than previous systems required.

2.3 Deep Learning Approaches

The shift from classical CAD methods to deep learning occurred for a few reasons. Firstly, deep learning models such as CNNs and ViTs have the ability to learn specific image features from pixel data directly rather than having to manually design features and rules for all types of lesions. This makes deep learning models more flexible and often more accurate across different diseases and imaging types. Secondly, the emerging of large labeled medical imaging datasets and the availability of powerful GPU computing have made it possible to train these deep networks effectively. Thirdly, deep learning networks are more robust to image quality, patient anatomy and scanner types compared to rule based systems. Finally, they continuously outperform classical CAD methods in pivotal studies. The subsequent sections review some of the popular deep learning models used in medical imaging research, and also used in this study.

2.3.1 nnU-Net

The nnU-Net addresses a persistent problem in biomedical image segmentation. Even well-established architectures like the U-Net require extensive, dataset-specific manual tuning of preprocessing, network topology, and training parameters. Small misconfigurations can cause large performance drops. Rather than proposing a new architecture, nnU-Net systematizes this configuration process itself. It decomposes design decisions into three categories:

1. Fixed parameters that generalize across datasets (e.g., architecture template, loss function, optimizer).

2. Rule-based parameters derived automatically from a "dataset fingerprint" (image size, spacing, intensity distribution, modality) via heuristics (e.g., target spacing, patch size, batch size, network depth).

3. A small set of empirical parameters selected via cross-validation (choice among 2D, 3D, and 3D cascade configurations, and whether to apply post-processing such as largest-component suppression).

Figure 2.2 shows the nnU-Net configuration method:

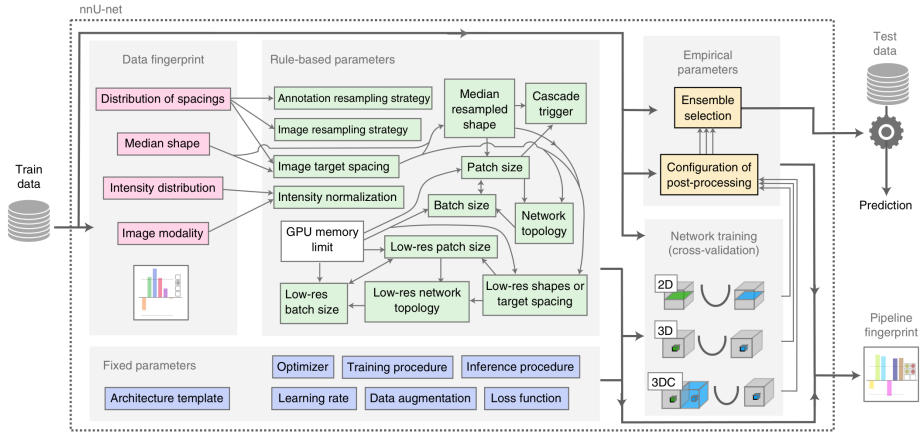

Figure 2.2: nnU-Net configuration method

Evaluated across 23 public datasets and 53 segmentation tasks spanning MRI, CT, electron microscopy, and fluorescence microscopy, nnU-Net outperformed most specialized, task-tuned pipelines without any manual intervention, setting a new state of the art on 33 of 53 target structures. Notably, the authors' analysis of the KiTS 2019 leaderboard showed that configuration choices such as patch size, spacing, normalization affected performance more than architectural modifications. Figure 2.3 shows example segmentations produced by nnU-Net on different datasets:

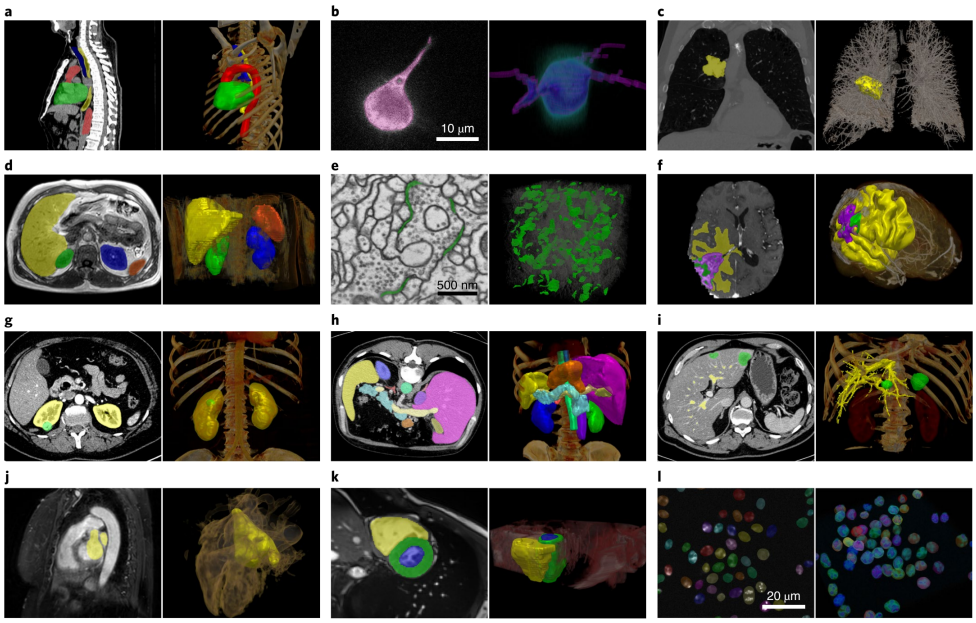

Figure 2.3: Example segmentations by nnU-Net

This suggests that careful pipeline configuration is often the primary bottleneck in medical image segmentation. This positions nnU-Net less as a competing architecture and more as a strong, reproducible baseline and out-of-the-box framework, and it is frequently cited in the literature as the standard benchmark against which new segmentation methods are compared.

2.3.2 BioMedParse

BiomedParse is a "Foundation Model for Joint Segmentation, Detection, and Recognition of Biomedical Objects Across Nine Modalities" [15]. Rather than relying on a fixed label set during inference, the model accepts a natural language prompt describing the target anatomy or pathology and produces a pixel level mask for the queried concept. This makes BiomedParse especially useful in settings where the structures of interest are heterogeneous, difficult to enumerate in advance, or described differently across institutions and reports.

The model combines image encoding with language understanding and uses a transformer based decoder to align the prompt with the visual features extracted from the scan [15]. Figure 2.4 shows the flowchart of the architecture:

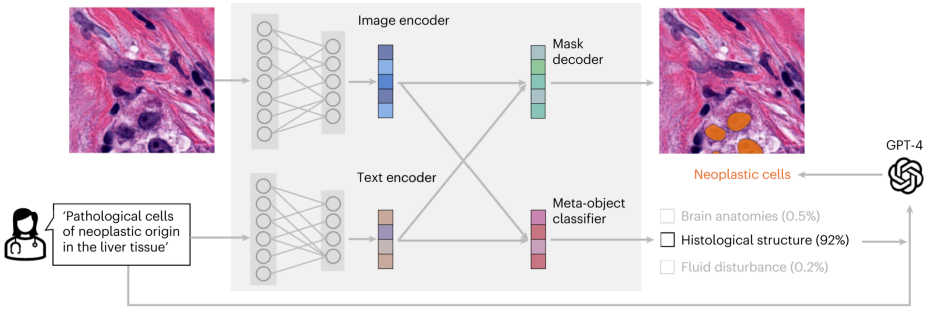

Figure 2.4: BiomedParse architecture

The output is a segmentation mask, as shown in Figure 2.5.

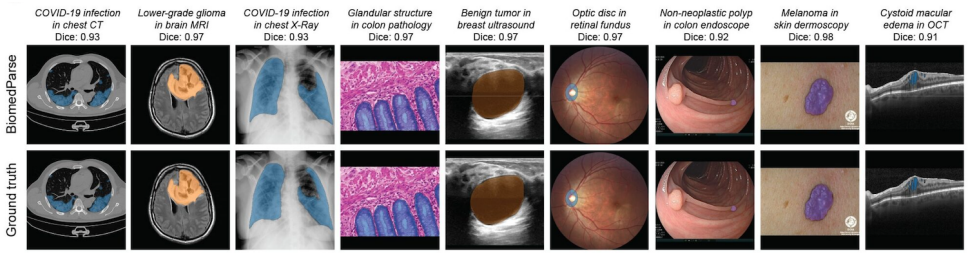

Figure 2.5: Example outputs of BiomedParse

BiomedParse serves as a SotA reference model for discussing localization quality, and interpretability in our study.

2.3.3 TotalSegmentator

Many deep learning methods were developed to segment only one or a few organs. Wasserthal et al. introduced TotalSegmentator, a deep learning framework that can automatically segment 104 anatomical structures from a single CT scan [16]. Figure 2.6 shows the main classes of TotalSegmentator, which include 27 organs, 59 bones, 10 muscles, 8 vessels.

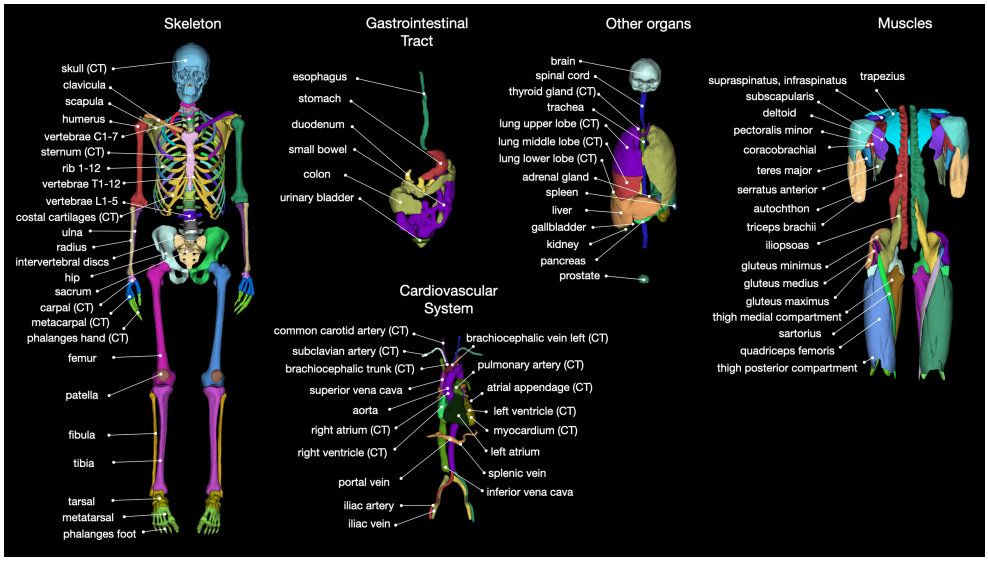

Figure 2.6: Main classes of TotalSegmentator

The framework is built on the nnU-Net architecture and was trained using more than 1,200 routine clinical CT scans collected from different hospitals and patient groups [16]. It achieved a mean Dice Similarity Coefficient (DSC) of about 0.94. Because of its high segmentation accuracy, TotalSegmentator is commonly used as a preprocessing tool in medical image research [17].

2.3.4 Merlin

Merlin is an AI model that can help interpret CT scans automatically. 3D vision-language model (VLM) built to understand abdominal CT scans by learning jointly from the images, radiology reports, and electronic health record (EHR) data [18].

Most earlier medical vision-language models were built for 2D images, such as chest X-rays, paired with short captions. CT scans are different, they are 3D volumes made of hundreds of slices, and simply applying a 2D model slice-by-slice throws away important 3D spatial information, like how a lesion connects across neighboring slices. Merlin was built to fix this by processing the full 3D CT volume directly, instead of treating it as a stack of unrelated 2D images [18].

Merlin is trained using over 6 million CT images from more than 15,000 scans, paired with over 1.8 million EHR diagnosis codes and more than 6 million tokens of radiology report text [18]. Instead of relying only on manually labeled data, which is slow and expensive to collect, Merlin learns from data hospitals already generate: doctors' reports and diagnosis codes. Merlin follows the concept introduced by CLIP, a foundational vision-language model designed originally for natural images. In CLIP style training, similar or matching image or text pairs closer to each other in a shared representation space while non-matching pairs are pushed apart making the model learn which images and text describe the same thing. Merlin adapts this idea to 3D CT data and adds EHR code supervision on top, which earlier medical VLMs generally did not use [18].

Merlin is part of a broader wave of work bringing vision-language modeling into 3D medical imaging, alongside similar efforts like CT-CLIP, which uses the same contrastive approach but focuses on chest CT scans[18]. Overall, Merlin shows that

learning directly from full 3D volumes and everyday hospital data, rather than small 2D datasets with manual labels, can produce useful for reducing radiologist workload.

2.3.5 MedSAM

MedSAM adapts the Segment Anything Model (SAM), originally developed by Meta AI for natural images, into a general-purpose segmentation model for medical scans [19]. Rather than training a separate network for each organ or modality, a single fine-tuned model is reused across many segmentation tasks. A key limitation is that MedSAM operates on 2D slices. Applying it to a 3D chest CT volume therefore requires a separate bounding box on every slice, and the resulting masks do not capture how a nodule connects across neighbouring slices.

MedSAM2 addresses this limitation by building on the SAM2 architecture and its memory-attention mechanism, allowing a single prompt to be propagated across an entire volume instead of being redrawn slice by slice, which substantially reduces the annotation effort required [20]. Specifically, that prompt is purely spatial, a box(two corner points), a point(roughly inside the target), or a mask(rough/full segmentation on that one slice) on one key slice, with no text or class label involved; from it, MedSAM2 segments the object on that slice and then propagates the mask forward and backward through the remaining slices via memory attention, without requiring a new prompt on each one.

Despite this improvement, pulmonary findings remain difficult for these models to segment accurately. A benchmarking study comparing MedSAM and MedSAM2 against standard segmentation networks found that both achieved strong performance on whole-lung segmentation but performed noticeably worse on individual tumours and small nodules, with results sensitive to the quality and consistency of the box prompts [21]. This reflects a broader limitation of prompt-driven segmentation models.

This sensitivity to prompt quality points to a more basic problem: neither MedSAM nor MedSAM2 can be told *what* to segment in words, simply because the SAM/SAM2 backbone they build on has no language pathway, only a geometric one. Meta's SAM 3 is aimed exactly at this gap through what it calls *Promptable Concept Segmentation*. Specifically, that prompt is purely spatial, a box(two corner points), a point(roughly inside the target), or a mask(rough/full segmentation on that one slice)t [22]. MedSAM-3 takes this a step further and fine-tunes the architecture on medical images paired with concept phrases, things like "lung infection" for chest X-ray or "pulmonary artery" for 3D lung CT [23]. Even so, the authors' own ablations make clear that text alone does not remove the need for a geometric anchor. On the 3D Parse2022 lung CT benchmark, raw SAM 3 given nothing but the phrase "pulmonary artery" managed a Dice of only 0.53, well short of nn-U-Net's 0.83; on 2D benchmarks such as BUSI, text-only prompting scored 0 for SAM 3 and just 0.27 after MedSAM-3 fine-tuning, jumping to 0.78 only once a bounding box was added alongside the text [23]. It is also worth noting that MedSAM-3's fine-tuning was limited to four 2D datasets, and its 3D lung CT evaluation went no further than the untuned SAM 3 baseline on pulmonary artery delineation; neither this paper nor the wider MedSAM family reports results on lung nodule or ground-glass-opacity segmentation in chest CT.

Figure 2.7: Architecture of MedSAM-3

2.3.6 CT-CLIP

CT images contain valuable information for diagnosing pulmonary diseases and are commonly used in many Deep Learning approaches as a single source of truth. However, radiology reports also provide detailed clinical descriptions of the findings in these CT images. Hamamci et al. proposed Computed Tomography Contrastive Language-Image Pre-training (CT-CLIP) [24], a vision-language foundation model for chest CT images, which utilizes both modalities. Unlike traditional supervised models that require large amounts of labeled data, CT-CLIP learns the relationship between CT scans and their corresponding radiology reports using contrastive learning. This allows the model to learn meaningful image and text representations without task-specific annotations.

CT-CLIP consists of a 3D image encoder and a text encoder. The image encoder extracts features from CT volumes, while the text encoder processes the corresponding radiology reports. Figure 2.8 shows the architecture of CT-CLIP.

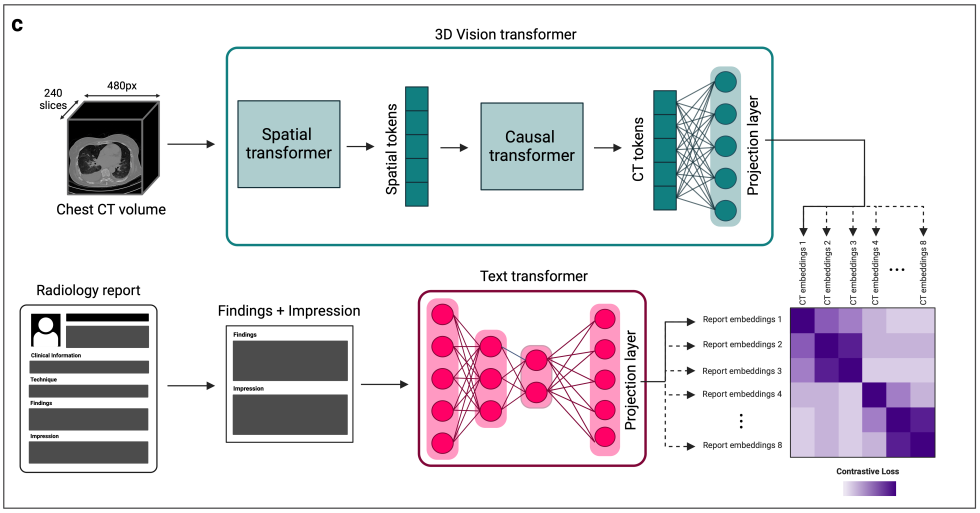

Figure 2.8: CT-CLIP architecture

During training, the model learns to match each CT scan with its correct report in a shared feature space. The model was trained using the CT-RATE dataset [25], which pairs over 50,000 reconstructed 3D chest CT volumes from around 25,700 scans of over 21,000 patients with their corresponding radiology reports. It has shown strong performance in pulmonary disease classification, image retrieval, and zero-shot learning tasks, outperforming fully supervised models across key metrics [24].

2.4 Explainable AI (XAI) in Medical Imaging

Many deep learning models achieve high accuracy, but their decision-making process is often difficult to interpret and are referred to as "black-box" models [26]. In healthcare, this lack of transparency can reduce clinicians' confidence in AI-assisted

diagnosis, as medical decisions directly affect patient outcomes. XAI methods aim to solve this issue.

2.4.1 LIME

Local Interpretable Model-agnostic Explanations (LIME), proposed by Ribeiro et al. [27], is a model-agnostic technique for explaining the predictions of any classifier or regressor by locally approximating its behavior with an interpretable surrogate model. The core idea is to treat the original model f as a black box and, for a given prediction on instance x, learn a simple explanation model g (typically a sparse linear model) that is faithful to f in the vicinity of x, even if g cannot approximate f well globally. Formally, LIME finds an explanation by minimizing an objective:

$$\xi(x) = \arg\min_{g} L(f, g, \pi_x) + \omega(g)$$

where L measures how unfaithful g is to f within a local neighborhood weighted by a proximity function π_x, and $\Omega(g)$ penalizes the complexity of the explanation to keep it human-interpretable. To ensure explanations are understandable regardless of the model's actual features, LIME operates over an interpretable representation such as a set of super-pixels for images—rather than the model's raw internal features (pixel tensors).

Through simulated experiments and studies, the authors demonstrate that LIME explanations are faithful to the underlying model, recovering ¿90% of known important features [27]. It helps users select the better-generalizing classifier between two competing models even when held-out accuracy is misleading, enables non-experts to perform effective feature engineering to improve an untrustworthy classifier, and allow users to detect models that rely on spurious correlations.

2.4.2 SHAP

SHapley Additive exPlanations (SHAP) is a unified framework for interpreting the predictions of complex machine learning models, proposed by Lundberg and Lee [28]. It builds on the observation that many existing interpretability methods—including Local Interpretable Model-agnostic Explanations (LIME), Deep Learning Important FeaTures (DeepLIFT), layer-wise relevance propagation, and classical Shapley value approaches—can all be framed as instances of a single class called additive feature attribution methods, in which an explanation model g approximates the original model f as a linear function of binary "presence/absence" indicator variables:

$$g(z') = \phi_0 + \sum \phi_i z'_i$$

Within this class, the authors show that only one explanation satisfies three desirable properties simultaneously:

1. Local Accuracy - The explanation must sum to match the model's actual output.
2. Missingness - Features absent from the input receive zero attribution.
3. Consistency - A feature's attributed importance cannot decrease if the model comes to rely on it more, holding other inputs fixed.

This unique solution fairly distribute a prediction's output among input features by averaging each feature's marginal contribution across all possible orderings/subsets of the other features—giving rise to the name SHAP values.

Because exact Shapley value computation is combinatorially expensive, the paper introduces several practical estimation methods. Kernel SHAP is a model-agnostic approach that computes SHAP values via specially weighted linear regression, connecting the LIME framework to Shapley values by deriving the specific loss function, weighting kernel, and regularization needed to make LIME's regression-based estimate satisfy the above three properties [28].

For deep neural networks specifically, the authors introduce Deep SHAP, which combines the compositional, layer-by-laye propagation style of DeepLIFT with Shapley value theory to more efficiently approximate feature attributions, and Max SHAP, which computes Shapley values for max-pooling operations in polynomial rather than exponential time. Empirically, the authors demonstrate that SHAP values align more closely with human-generated explanations than LIME or DeepLIFT, and that Kernel SHAP achieves comparable accuracy to sampling-based Shapley estimation with substantially fewer model evaluations, establishing SHAP as a theoretically grounded and practically efficient tool for model interpretability [28].

2.4.3 Grad-CAM

Gradient Class Activation Mapping (Grad-CAM), proposed by Selvaraju et al. [29], is a technique for producing visual explanations from CNNs by using gradient information to identify which regions of an input image are important for a given prediction. Unlike its predecessor, Class Activation Mapping (CAM), Grad-CAM works with any CNN-based architecture without changes or retraining. The method computes the gradient of a target class score with respect to the feature maps of the last convolutional layer, then global-average-pools these gradients across the spatial dimensions to obtain a set of neuron importance weights α_k^c. A weighted combination of the forward activation maps using these weights are passed through a ReLU to retain only features with a positive influence on the target class. This produces a coarse heatmap highlighting class-discriminative regions of the image. The authors formally prove that Grad-CAM is a strict generalization of CAM, reducing to the same weights when applied to CAM-compatible architectures.

Through extensive evaluation, the authors demonstrate that Grad-CAM achieves superior weakly-supervised localization performance compared to prior methods without sacrificing classification accuracy [29]. They further showed that Grad-CAM's localizations correlate more strongly with occlusion-based faithfulness measures than competing visualization techniques, indicating it is more faithful to the underlying model. Beyond diagnostic use for uncovering CNN failure modes and robustness to adversarial perturbations, the authors demonstrate Grad-CAM's utility for identifying and mitigating dataset bias, famously showing that a biased "doctor vs. nurse" classifier had learned to rely on gender-correlated cues such as hairstyle, rather than clinically relevant features.

2.4.4 Other CAM Methods

Grad-CAM++

Chattopadhyay et al. propose Grad-CAM++ [30] as a generalization of Grad-CAM that addresses two of its key limitations:

1. Degraded localization when multiple instances of the same class appear in a single image
2. Incomplete coverage of an object's extent in single-object images

While gradient-based methods like Grad-CAM provide fine-grained explanations, their performance drops when localizing multiple occurrences of the same class, and Grad-CAM heatmaps often fail to capture the entire object [30]. Grad-CAM++ uses a weighted combination of the positive partial derivatives of the last convolutional layer's feature maps with respect to a specific class score as pixel-wise weights, producing a visual explanation for the class of interest. By deriving closed-form higher-order gradient weighting coefficients rather than Grad-CAM's simple global-average-pooled gradients, Grad-CAM++ generates explanation maps that better localize multiple object instances and cover objects more completely.

Score-CAM

Score-CAM proposed by Wang et al. [31] removes the dependence on gradients entirely. Unlike previous CAM approaches, Score-CAM obtains the weight of each activation map through its forward-passing score on the target class rather than through gradients, with the final saliency map obtained as a linear combination of these weights and the activation maps. Concretely, each channel of the final convolutional layer's activation map is upsampled to input resolution and used as a mask over the original image; the resulting masked images are forward-passed through the network, and the change in the target class's confidence score serves as that channel's importance weight. Because it dispenses with backpropagated gradients—which can be noisy or saturate—Score-CAM was shown to produce better visual quality and fairness in explanation than gradient-based CAM variants, at the cost of requiring multiple forward passes.

LayerCAM

Jiang et al. introduced LayerCAM [32] to address the coarse spatial resolution of standard CAM/Grad-CAM maps, which are typically generated only from the final convolutional layer. Because such maps have low spatial resolution, they often only locate coarse regions of target objects, limiting performance on tasks needing pixel-accurate localization. By rethinking the relationship between feature maps and their corresponding gradients, LayerCAM produces reliable class activation maps at different layers of the CNN, using element-wise gradient weighting, rather than globally pooled gradients, so that shallower layers—which retain finer spatial detail—can also contribute meaningful, class-discriminative localization. These layer-wise maps can then be fused into a single higher-resolution activation map, better capturing fine object boundaries than Grad-CAM's coarse, final-layer-only heatmap.

2.4.5 Limitations of XAI Methods

Despite substantial progress, XAI continues to face a fundamental accuracy-interpretability tradeoff: highly interpretable models such as decision trees, linear regression, and rule-based learners are intrinsically limited in their ability to model non-linear, high-dimensional relationships. Deep neural networks that achieve state-of-the-art performance remain the least interpretable [33]. Model extraction and distillation techniques, which attempt to approximate black-box behavior with a simpler surrogate model, partially address this but reduces fidelity to the original model's decision boundaries. Post-hoc explanation methods such as LIME offer only local fidelity and provide an unsatisfactory global approximation, and saliency maps such as Grad-CAM can produce noisy uninterpretable outputs. Furthermore, saliency maps assume that all model-relevant features are inherently human-interpretable, which does not always hold [34]. Explanation techniques are also vulnerable to manipulation and adversarial perturbation; LIME has been shown to develop self-learned patterns that produce misleading explanations under such conditions, raising concerns about the robustness and trustworthiness of the interpretations themselves [34].

Beyond technical constraints, XAI suffers from a lack of standardized, universally accepted evaluation frameworks, since interpretability and explainability are frequently conflated and no consensus exists on how to rigorously quantify explanation quality [35, 36]. Existing evaluation paradigms: application-grounded, human-grounded, and functionally grounded, each carry tradeoffs between cost, human involvement, and validity. The subjective, user-dependent nature of what constitutes a "good" explanation makes benchmarking especially difficult [33]. Additional challenges include the computational expense of methods like Model Class Reliance, the difficulty such methods have scaling to complex, high-dimensional, or strongly feature-correlated models, and the broader absence of a unified theoretical framework connecting the disparate strands of explanation research [37].

2.5 Sparse Coding

Why is sparsity useful?

How is sparse representation learned?

Sparse coding is a signal representation framework in which a signal is expressed as a linear combination of only a small number of elements ("atoms") drawn from a dictionary, rather than as a dense combination over an orthogonal basis. Formally, given a signal x and a dictionary D whose columns are atoms, sparse coding seeks a coefficient vector α such that:

$$x \approx D\alpha,$$

with α containing mostly zero entries. A key aspect of this framework is the use of overcomplete representations, where the dictionary D has more atoms than the dimensionality of the signal itself (i.e., more columns than rows). This redundancy is deliberate: it gives the representation greater flexibility to match the underlying structure of the signal, allowing different signals to be well-approximated by different small subsets of atoms, which improves representational power, robustness to noise, and the ability to capture diverse structural patterns compared to a complete (non-redundant) basis. The trade-off is that recovering α becomes an underdetermined

and generally NP-hard combinatorial problem, since many possible coefficient vectors could reconstruct x, and one must impose sparsity as a constraint or penalty to select a meaningful solution.

Because exact sparsest-solution search is computationally intractable, two main families of practical algorithms are used:

1. Greedy Algorithms (Matching Pursuit, OMP)
2. Convex relaxation Algorithms (LARS, FISTA)

Greedy methods iteratively select the atom most correlated with the current residual, add it to the active support set, and update the coefficients, repeating until a stopping criterion (sparsity level or residual error) is met; these methods are fast and simple but only approximate the optimal solution.

Convex relaxation methods instead replace the intractable $\ell 0$-norm (which counts nonzero entries) with the $\ell 1$-norm, yielding a convex optimization problem—as in Basis Pursuit or the Lasso—that can be solved reliably and, under certain conditions on the dictionary (e.g., restricted isometry or incoherence properties), provably recovers the same sparse solution as the original $\ell 0$ problem. In practice, greedy methods are often preferred for speed and low-sparsity regimes, while convex relaxation approaches tend to offer stronger theoretical guarantees and more stable solutions in noisier or higher-sparsity settings.

2.5.1 Matching Pursuit (MP)

Matching Pursuit is a greedy algorithm for approximating a signal x as a sparse combination of atoms from a dictionary D. It proceeds iteratively: at each step, it computes the inner product between the current residual (initially the signal itself) and every atom in the dictionary, selects the atom with the highest absolute correlation, and updates the coefficient for that atom by projecting the residual onto it. The residual is then updated by subtracting this contribution, and the process repeats until a stopping criterion is reached (target sparsity or residual energy threshold). MP is simple and computationally cheap, but because it only updates one coefficient per iteration without adjusting previously selected coefficients, it can select the same atom multiple times and converges slowly, and its solutions are generally suboptimal compared to more refined methods.

2.5.2 Orthogonal Matching Pursuit (OMP)

OMP improves on MP by re-optimizing all currently selected coefficients at every iteration rather than only the most recently added one. At each step, it selects the atom most correlated with the residual (as in MP), adds it to the active support set, and then solves a least-squares problem over all atoms currently in the support to obtain new coefficients that minimize the residual jointly. Because the residual is then orthogonal to all previously selected atoms, OMP never reselects the same atom twice, and it typically converges in fewer iterations and yields more accurate reconstructions than plain MP. The added cost is the least-squares solve at each iteration, making OMP more computationally expensive per step, though still tractable for moderate sparsity levels.

2.5.3 Least Angle Regression (LARS)

LARS is an efficient algorithm for computing the entire regularization path of the Lasso ($\ell 1$-penalized least squares) problem. Rather than fixing a penalty parameter λ and solving a single optimization, LARS starts with all coefficients at zero and iteratively identifies the predictor (atom) most correlated with the current residual, moving the coefficient in that predictor's direction. Instead of moving all the way to the least-squares solution as in stepwise regression, LARS moves only until another predictor becomes equally correlated with the residual, at which point that predictor is added to the active set and the direction is updated to remain equiangular between all active predictors. With a simple modification (allowing coefficients to be dropped from the active set when they cross zero), LARS exactly reproduces the full Lasso solution path with a computational cost comparable to ordinary least squares, making it a popular and efficient tool for convex-relaxation-based sparse coding, particularly when the full path of solutions across sparsity levels is of interest.

2.5.4 Fast Iterative Shrinkage-Thresholding Algorithm (FISTA)

FISTA solves $\ell 1$-regularized sparse coding problems, where the goal is to minimize:

$$\sqrt{\|x - D\alpha\|}^2 + \lambda\|\alpha\|_1 \tag{2.1}$$

It builds on the Iterative Shrinkage-Thresholding Algorithm (ISTA), which alternates between a gradient descent step on the data-fidelity term and a soft-thresholding (shrinkage) step that enforces sparsity by shrinking small coefficients toward zero. While ISTA converges at a sublinear rate of $O(1/k)$, FISTA accelerates this by incorporating a momentum term—extrapolating using a weighted combination of the current and previous iterates—which improves the convergence rate to $O(1/k^2)$ without increasing per-iteration cost. This makes FISTA well-suited to large-scale sparse coding and compressed sensing problems where fast, reliable convergence to the $\ell 1$-regularized solution is needed.

2.6 Dictionary Learning

Dictionary learning is a technique in which an image patch or a region of a medical scan is represented as a combination of a small number of basic building blocks, rather than by describing every pixel value directly. This idea comes from sparse coding theory and is widely used in image denoising, compression, and interpretable machine learning, including medical image analysis [38] [39].

A dictionary is a matrix, written as $D \in \mathbb{R}^{n\times k}$, where each column d_i is called an atom. Each atom is a small, fixed pattern, for example a short edge, a texture, or a simple shape, that acts like a basic building block for describing images [38].

$$D = [\, d_1, d_2, \ldots, d_k \,], \qquad \|d_i\|_2 = 1 \ \forall i \tag{2.2}$$

Each atom is normalized to unit length so that none of them dominate because of scale. A good dictionary has atoms that are simple and reusable.

Once a dictionary is available, any image $x \in \mathbb{R}^n$ is approximated as a weighted sum of a small number of atoms:

$$x \approx D\alpha = \sum_{i=1}^{k} \alpha_i d_i \tag{2.3}$$

Here, $\alpha \in \mathbb{R}^k$ is the sparse code; most of its values are zero, and only a few atoms are actually used to rebuild the image. Finding this code is called sparse coding, and it can be solved by minimizing the reconstruction error together with a sparsity penalty [39] [40]:

$$\min_{\alpha} \ \frac{1}{2}\|x - D\alpha\|_2^2 + \lambda\|\alpha\|_1 \tag{2.4}$$

This is the Lasso formulation, where λ controls how sparse the solution is [41]. When the dictionary itself is learned from training data $X = [x_1, \ldots, x_m]$, both D and the codes $A = [\alpha_1, \ldots, \alpha_m]$ are optimized together

$$\min_{D,A} \ \|X - DA\|_F^2 + \lambda\|A\|_1 \quad \text{subject to} \quad \|d_i\|_2 = 1 \tag{2.5}$$

Learned atoms are not random they turn out to look like meaningful patterns. Scince each image patch is reconstructed using only a small number of atoms. Each atoms tends to specialize in representing a simple pattern rather than blending many patterns together. This lets a person actually look at a lerned dictionary and understand what each atom represents, wich is must harder with internal weights of a deep neural network. In medical imaging, atoms can end up representing specific tissue textures or lesion patterns that a radiologist can vissually recognize[42].

Learning a dictionary is usually done by fixing the dictionary and solving for sparse codes, then fixing the codes and updating the dictionary. The sparse coding step is commonly solved with Orthogonal Matching Pursuit (OMP), a greedy method that adds one atom at a time [43]

$$i^* = \arg\max_i \ |d_i^T r|, \qquad r = x - D\alpha \tag{2.6}$$

or with Lasso/LARS, using the ℓ_1-penalized objective shown above [41]. The dictionary update step is where the main algorithms differ, covered below.

2.6.1 K-Means Singular Value Decomposition (K-SVD)

K-SVD is one of the most widely used algorithms for learning a dictionary, and it solves the following optimization problem[41]

$$\min_{D,X} \left\{\|Y - DX\|_F^2\right\} \quad \text{subject to} \quad \forall i, \ \|x_i\|_0 \le T_0 \tag{2.7}$$

Here, Y is the matrix of training data, for example CT image patches, D is the dictionary being learned, and X is the matrix of sparse codes, where each column x_i is the code for one training sample. The constraint $\|x_i\|_0 \le T_0$ means that each code is allowed to use at most T_0 atoms, using the ℓ_0 norm to directly count non-zero entries rather than approximating sparsity with an ℓ_1 penalty like Lasso does. This gives K-SVD tighter control over exactly how sparse each representation is.

K-SVD solves this problem by alternating between two steps. First, with the dictionary D fixed, it finds the sparse codes X using a pursuit algorithm such as Orthogonal Matching Pursuit. Second, with the codes fixed, it updates the dictionary one atom at a time instead of all at once, which makes it converge faster and produce sharper, more specific atoms [41].

2.6.2 Method of Optimized Directions (MOD)

Method of Optimized Directions is an earlier and simpler approach than K-SVD. Instead of updating atoms one at a time, MOD updates the entire dictionary at once, using a direct least-squares solution [**?**]. Given the training data X and current sparse codes A, the optimal dictionary is:

$$D = XA^T(AA^T)^{-1} \tag{2.8}$$

This comes directly from setting the derivative of the reconstruction error $\|X - DA\|_F^2$ to zero and solving for D. MOD is fast and easy to implement, but because it updates all atoms together, it tends to need more iterations to converge compared to K-SVD, and can get stuck producing less sharp atoms [**?**].

2.6.3 Online Dictionary Learning (ODL)

Both K-SVD and MOD assume the entire training dataset fits in memory and is processed all at once, which becomes impractical for very large image datasets. Online Dictionary Learning solves this by processing the data in small batches, updating the dictionary incrementally as new samples arrive, instead of waiting to see the whole dataset [**?**]. At each step t, a new sample x_t is drawn, its sparse code α_t is computed using the current dictionary, and then the dictionary is updated using accumulated statistics from all samples seen so far:

$$A_t = A_{t-1} + \alpha_t \alpha_t^T, \qquad B_t = B_{t-1} + x_t \alpha_t^T \tag{2.9}$$

$$D_t = \arg\min_D \frac{1}{t} \sum_{i=1}^{t} \left(\frac{1}{2} \|x_i - D\alpha_i\|_2^2 \right) \tag{2.10}$$

which can be solved efficiently using A_t and B_t instead of storing all past samples. This makes ODL well suited for large-scale problems, including learning dictionaries from thousands of medical image patches, since memory use stays constant regardless of dataset size [**?**].

2.7 Discriminative Dictionary Learning

Traditional dictionary learning methods mainly focus on minimizing the reconstruction error between the input data and its sparse representation [44]. However, good reconstruction does not necessarily produce features that are useful for classification. Samples from different classes may activate similar atoms if the learned dictionary only captures common structures in the data.

Discriminative dictionary learning addresses this limitation by incorporating class information during dictionary optimization. The objective is not only to represent the input samples accurately but also to learn sparse codes that improve the separation between different classes. By learning class aware atoms, discriminative dictionary learning can provide both classification capability and a more interpretable representation. Several approaches have been proposed, including Label Consistent K-SVD (LC-KSVD) and Fisher Discriminative Dictionary Learning (FDDL), which extend the K-SVD framework by introducing supervised learning constraints.

2.7.1 Label Consistent K-SVD (LC-KSVD)

Label Consistent K-SVD (LC-KSVD), introduced by Jiang et al. [45], extends the classic K-SVD framework by embedding label consistency into the dictionary update process. The core idea is to encourage sparse codes from samples of the same class to exhibit similar activation patterns, while those from different classes diverge. This is achieved without sacrificing the algorithm's efficient iterative structure involving sparse coding and dictionary refinement.

The method comes in two main variants. LC-KSVD1 incorporates a label consistency term that aligns the sparse codes with an ideal discriminative structure through a linear transformation:

$$\min_{D,A,A_c} \|X - DA\|_F^2 + \alpha\|Q - A_cA\|_F^2 \quad \text{s.t.} \quad \|a_i\|_0 \leq T \ \forall i. \tag{2.11}$$

Here, Q is a target matrix encoding class specific indicator patterns, and A_c learns to map codes toward this structure. LC-KSVD2 extends LC-KSVD1 by adding a classifier learning term. The dictionary, sparse codes, label consistency transformation, and classifier are jointly optimized:

$$\min_{D,A,A_c,W} \|X - DA\|_F^2 + \alpha\|Q - A_cA\|_F^2 + \beta\|H - WA\|_F^2 \quad \text{s.t.} \quad \|a_i\|_0 \leq T, \tag{2.12}$$

where H contains the label information. The additional classification term allows the learned sparse representation to be directly optimized for prediction. Due to its ability to learn class specific atoms, LC-KSVD has been applied in image classification tasks where interpretability of learned features is important [45].

2.7.2 Fisher Discriminative Dictionary Learning (FDDL)

Fisher Discriminative Dictionary Learning (FDDL), proposed by Yang et al. [46], is another supervised dictionary learning approach that directly enforces Fisher like criteria on both the dictionary atoms and the sparse coefficients. It learns a structured dictionary $D = [D_1, D_2, \ldots, D_c]$ with class specific sub dictionaries D_i while promoting sparse codes A that exhibit low within class scatter and high between class scatter.

The core objective combines a discriminative fidelity term $r(\cdot)$ (ensuring good reconstruction within class and poor reconstruction across classes) with a Fisher discrimination penalty on coefficients:

$$\min_{D,X} \sum_i r(A_i, D, X_i) + \lambda_1\|X\|_1 + \lambda_2\Big(\operatorname{tr}(S_W(X)) - \operatorname{tr}(S_B(X)) + \eta\|X\|_F^2\Big), \tag{2.13}$$

where $r(\cdot)$ encourages each class to be well represented by its own sub dictionary while poorly represented by others. S_W and S_B denote the within class and between class scatter matrices of the sparse codes. The elastic term controlled by η is added to guarantee that the overall Fisher discrimination function remains convex, which greatly improves optimization stability [46].

Compared with LC-KSVD, which introduces label consistency through a learned transformation and classifier, FDDL focuses on improving the statistical distribution of sparse codes. This makes it suitable for classification tasks where the separation between classes is important.

2.7.3 Frozen Dictionary Learning

While most discriminative methods update the entire dictionary during training, certain scenarios benefit from preserving previously acquired knowledge. Frozen dictionary learning, as introduced in the Outlier Learning via Augmented Frozen (OLAF) Dictionaries framework [47], addresses this by fixing a subset of atoms learned from baseline (normal) data while allowing new atoms to adapt to anomalous samples.

The composite dictionary takes the partitioned form $D = [D_f \mid D_u]$, where D_f remains frozen after initial training on normal data, and only the unfrozen portion D_u is updated on data containing deviations. Both K-SVD and Alternating Minimization algorithms can be modified straightforwardly to respect this constraint by skipping updates on frozen columns during the dictionary refinement stage [47].

This approach is particularly useful when preserving existing knowledge is important. In medical imaging, a dictionary learned from normal anatomy can be kept fixed, while additional atoms are learned from abnormal cases. This separation helps maintain normal structural information and improves the interpretability of disease related atoms. However, the effectiveness of this method depends on the quality of the initial frozen dictionary, since an incomplete initial representation may limit the ability of the model to adapt to new patterns.

2.8 Dictionary Learning in Medical Imaging

Dictionary learning has become a powerful tool across the medical imaging pipeline, offering data-adaptive alternatives to fixed transforms such as DCT or wavelets by learning sparse representations directly from image patches [48]. We have identified that DL-based methods are applied to medical imaging in 5 key aspects:

1. Signal Denoising
2. Image Reconstruction
3. Super-Resolution
4. Lesion Detection and Classification
5. Segmentation

2.8.1 Signal Denoising

In denoising, DL-based methods learn a dictionary of atoms from the noisy image itself, then reconstruct a clean image by enforcing that each local patch be sparsely represented by these learnt atoms. This approach has been applied to remove Gaussian noise from natural images [49]. In medical imaging, sparse denoising in diffusion MRI has shown to outperform standard DCT/FFT-based denoising by adapting to the specific structure of the data [50]. Shi et al. proposed an artifact suppressed dictionary learning algorithm (ASDL) [51], which exploited scale and orientation to build discriminative dictionaries for artifact suppression in LDCT.

2.8.2 Image Reconstruction and Super-Resolution

In reconstruction, DL has addressed the inverse problem of recovering images from undersampled k-space acquisitions in compressed sensing MRI [52]. Patch-based dictionaries provide a sparser and more flexible representation than global transforms like wavelets or temporal Fourier bases, enabling more accurate recovery from highly undersampled data.

Closely related to reconstruction is super-resolution, where coupled high-resolution and low-resolution dictionaries are co-trained so that corresponding patches share the same sparse code. This allows high-resolution image patches to be hallucinated from low-resolution inputs. This has been used to enhance cardiac MRI slabs and single-image brain MRI without requiring accurate subpixel alignment [53].

2.8.3 Detection, Classification and Segmentation

Since standard reconstructive dictionaries lack discriminative power, DDL techniques can be used for detection, classification and segmentation. DDL incorporates class labels during training, either by learning separate class-specific dictionaries and using representation residuals for classification, or by jointly learning a shared dictionary alongside a linear classifier over the sparse coding coefficients [48].

Class-specific dictionary learning has been directly applied to CAD for cancer detection. Liu et al. [54] proposed a sparse representation-based classification framework for colorectal polyp and lung nodule detection. The approach learned separate overcomplete dictionaries per class via K-SVD and classifying test samples either by the class whose dictionary yields the most non-zero coefficients when pooled into a global dictionary, or by the class-specific dictionary that achieves the sparsest reconstruction. This generative approach naturally extended to multi-class problems and, when fused with a relevance vector machine baseline through a gated decision tree, achieved consistent sensitivity gains at clinically acceptable false-positive rates on large multi-site colon and lung datasets.

Wu et al. [55] extended class-specific dictionary learning to pulmonary nodule detection in CT, addressing the practical problem that accurate nodule segmentation is often unreliable due to irregular shapes and low contrast. Rather than relying on precisely segmented ROIs, they learned two classification-oriented sub-dictionaries (nodule and non-nodule) together with a shared background dictionary, following the DL-COPAR formulation [56]. Sparse coefficients from the class-specific sub-dictionaries were pooled into feature vectors, refined using DX-score-based feature selection, and passed to an SVM classifier, yielding good classification performance on the LIDC-IDRI dataset without requiring accurate nodule segmentation, and demonstrating that discriminative dictionary learning can substitute for precise localization as a prerequisite for feature extraction.

Ben-Cohen et al. [57] extended dictionary learning-based approaches from classification into a full lesion detection pipeline, combining a global fully convolutional network (FCN) that generates a lesion probability map across the CT volume with a local, discriminative dictionary-based module for false-positive reduction. Candidate regions from the FCN were partitioned into SLIC uperpixels, each represented by a 125-dimensional textural feature vector, and classified using LC-KSVD. A further innovation was online, per-case fine-tuning of the dictionary and linear classifier using high and low-confidence superpixels drawn from each test case's own FCN

probability map, personalizing the discriminative dictionary without additional manual labels. On a clinical liver metastases dataset, this combined FCN-plus-sparsity framework achieved 94.6% sensitivity at 2.9 false positives per case, outperforming the FCN alone, U-Net, a superpixel-only sparse classifier, and a patch-based CNN, with particularly strong performance maintained on small lesions.

Al-Shaikhli et al. extended dictionary learning to a unified multi-class classification and multi-label segmentation framework for brain tumors. It is built around two distinct K-SVD dictionaries learned from a single Flair-MRI modality. A feature dictionary, learned from global topological features combined with gray-level co-occurrence texture features, is used for classification of the volume as normal or abnormal. If abnormal, it is further classified by finding which class-specific dictionary yields the sparsest representation of the test features. Separately, coupled data-label dictionaries are learned jointly from voxel-wise gray-scale patches and their corresponding ground-truth segmentation labels, so that each atom in the image dictionary has an associated atom in the label dictionary; at test time, sparse similarity between test patches and the data dictionary retrieves the matching label-dictionary atoms, which supply foreground/background costs for multi-label graph-cut segmentation of necrosis, edema, and enhancing/non-enhancing tumor. This decoupling of global topological/textural features for classification from local voxel-wise coupled dictionaries for segmentation, using only a single MRI channel, achieved 94.26% classification accuracy and segmentation performance competitive with or exceeding multi-channel BraTS-2013 methods requiring four MRI modalities.

Anitha's more recent hybrid framework similarly separates dictionary-based segmentation from a distinct classification stage, but replaces reconstruction-residual dictionary classification with a downstream regression-based classifier and shifts from pure MRI to fused 3D MRI-CT input: a Sparse Mahalanobis Dictionary learns class-discriminative atoms for tumor versus non-tumor tissue using a Mahalanobis-distance nonlinearity rather than plain Euclidean sparse coding. These dictionary-derived features feed a Multimodal Markov Random Forest that labels mutually exclusive and exhaustive tissue regions to avoid the tissue loss common in multi-decomposition fusion pipelines. The segmented tumor regions are then classified further by extracting nested 3x3 patches, computing Spearman-correlation-based co-occurrence features, and passing them to a Lagrangian neural network. On combined CT-hemorrhage and MRI tumor datasets this pipeline reported 99.7% accuracy, 98.9% sensitivity, and 98.7% specificity, outperforming CNN, CapsNet, ELM, and SVM-based baselines.

Wang et al. extended coupled dictionary learning to a multi-modality setting for nasopharyngeal carcinoma (NPC) segmentation, addressing the practical constraint that clinical datasets often mix multi-modality patients (with both CT and MRI) and single-modality patients. Building on local coordinate coding, their method jointly learns three coupled dictionaries (CT, MRI, and label) by sharing sparse coefficients across CT and MRI patches for multi-modality samples while separately fitting single-modality patches to their respective dictionary-label pairs, allowing all available data to contribute to a common sparse-coding space rather than discarding incomplete cases. At test time, a voxel is labeled by jointly encoding its CT and MRI patches against the learned CT/MRI dictionaries and projecting the resulting sparse coefficients onto the label dictionary. This joint approach outperformed single-modality dictionary learning (DSC of 76.87% vs. 65.36% for CT-only and 74.94% for MRI-only)

and achieved segmentation accuracy comparable to prior specialized NPC methods.

Dong et al. proposed a simultaneous CT reconstruction and segmentation (SRS) framework that uses a discriminative dictionary pair, one for representing image intensity patches and one for representing the corresponding class-probability patches, learned jointly via an extension of K-SVD. The two dictionaries share the same sparse codes for each patch, so that during reconstruction the sparse coding step simultaneously constrains both the intensity reconstruction and the segmentation. The segmentation acts as a built-in regularizer for the otherwise ill-posed inverse problem. Their optimization alternates between sparse coding, an intensity reconstruction update solved via CGLS, and a class-probability update solved with a Frank-Wolfe algorithm, with total-variation regularization on the class probabilities to encourage piecewise-constant, smooth class interfaces. Tested on textured Brodatz-based phantoms and simulated fibrous material samples under varying numbers of projection angles, the method outperformed a non-dictionary SRS baseline and two weaker dictionary-based pipelines, with the gap widening as the number of projections decreased. The authors note the method's reliance on training data that adequately spans the class-interface geometries expected in the target images, since interfaces not represented in the dictionary tend to be oversmoothed or straightened in the output.

A related work applies DDL to the segmentation problem, once images are already reconstructed. Tong et al. extended their earlier hippocampus-labeling framework into a discriminative dictionary learning for segmentation (DDLS) method capable of handling multiple structures simultaneously. Their approach jointly learns a reconstructive dictionary and a linear classifier from patches extracted out of a pool of selected atlas images, so that the sparse code of a target patch can be mapped, via the learned classifier, directly to a probabilistic label vector. This produces a subject-specific probabilistic atlas that is then refined with a graph-cuts post-processing step. A central contribution is a voxel-wise local atlas selection strategy (L-DDLS), which selects different subsets of atlases at different spatial locations based on local similarity, rather than relying on one globally selected atlas set (G-DDLS) or a fixed offline-trained dictionary (F-DDLS). This addresses the high inter-subject anatomical variability of abdominal organs, particularly the pancreas, which is poorly served by whole-image similarity measures dominated by large organs like the liver. A multi-resolution Gaussian-pyramid scheme was also incorporated to keep the patch-based approach computationally tractable for large 3D organs. Evaluated on 150 abdominal CT scans, L-DDLS achieved Dice overlaps of 94.9%, 93.6%, 71.1%, and 92.5% for liver, kidneys, pancreas, and spleen respectively, consistently outperforming majority-voting label fusion, non-local patch-based segmentation, and both G-DDLS and F-DDLS variants, with the largest relative gains for the anatomically variable pancreas. Notably, these results were obtained using only affine (not deformable) registration, underscoring how a learned discriminative dictionary can substitute for some of the burden typically placed on accurate non-rigid registration in multi-atlas segmentation pipelines.

Farhangi's doctoral thesis develops a shape-based segmentation framework for lung nodules in thoracic CT that relies on a dictionary of previously delineated nodule shapes rather than intensity patches. The method, called Sparse Combination of Training Shapes (SCoTS), embeds nodule boundaries as zero level sets of signed distance functions (SDFs) and builds a dictionary by vectorizing SDFs from a training set of expert-delineated nodules. The evolving segmentation surface is then approx-

imated at each iteration as a sparse linear combination of these dictionary atoms, solved via an l1-penalized least-squares formulation using a coordinate-wise "shooting" algorithm. The key contribution is coupling this sparse shape prior with a Chan-Vese region-based active contour. The relative influence of the shape prior versus the low-level image-intensity energy is adjusted dynamically at each iteration based on the sparsity of the reconstruction coefficients, so that as the evolving surface approaches a plausible nodule shape and its representation becomes sparser, the shape prior is trusted more, preventing the contour from leaking into adjacent tissues of similar density. Validated on 542 3D nodules from the LIDC-IDRI database against four independent radiologists, SCoTS achieved a mean DSC of about 0.71-0.72 across radiologist delineations, with the primary failure mode occurring for nodules whose shapes were poorly represented in the training dictionary.

Discriminative dictionary learning has also been extended to multi-modal neuroimaging classification of Alzheimer's disease (AD) and mild cognitive impairment (MCI). Li et al. built on their previously introduced supervised within-class-similarity discriminative dictionary learning (SCDDL) algorithm by adding a within-class-similarity term to the standard discriminative dictionary objective, which penalizes deviation of each sample's sparse code from its class mean, alongside the usual reconstruction and linear classification error terms. To handle multiple imaging modalities, they extended SCDDL into a multi-modality version (mSCDDL) that learns a separate dictionary and classifier per modality and fuses per-modality classifier outputs through a weighted combination, with weights optimized by grid search. Using ROI-averaged features from 113 AD, 110 MCI, and 117 normal-control subjects from ADNI, mSCDDL achieved 97.4% accuracy for AD versus normal and 77.7% accuracy for MCI versus normal, outperforming each single-modality SCDDL variant as well as other established multi-modal fusion methods, while requiring substantially less computation time, demonstrating that a compact learned dictionary of just 20 atoms can rival methods that use the entire training set as an implicit dictionary.

The same research group subsequently pushed this framework into a kernelized setting to better exploit cross-modal complementarity. Li et al. introduced multi-feature kernel SCDDL (MKSCDDL), which maps each modality's features into a higher-dimensional space via Mercer kernels before applying the SCDDL objective, expressing the dictionary implicitly as a linear combination of kernel-mapped training samples. This allows the same reconstruction, classification-error, and within-class-similarity terms can be optimized without explicitly forming the high-dimensional mapping. Sub-kernels for each modality are then combined, extending the earlier weighted-combination scheme of mSCDDL into a single unified kernel objective. Evaluated on the same three-modality ADNI cohort, MKSCDDL improved classification accuracy over mSCDDL and other multi-modal baselines across all three diagnostic contrasts: 98.2% for AD vs. cognitively unimpaired (CU), 78.5% for MCI vs. CU, and 74.5% for AD vs. MCI, while also reducing test time.

Most relevant to our study, Liyanage et al. tackle COVID-19 detection from chest CT by combining two complementary DL strategies rather than a single dictionary. The first, frozen dictionary learning, addresses class imbalance in the training data, where normal lung-field patches vastly outnumber abnormal patches by learning dictionary atoms hierarchically. The second, LC-KSVD, learns a discriminative dictionary directly from patient-level class labels (normal, common pneumonia, novel coronavirus pneumonia). The two dictionaries are concatenated into a single aug-

mented dictionary, patches are sparse-coded against it via OMP, and simple per-scan statistics (mean, variance, non-zero count) of the sparse coefficients associated with each class-relevant dictionary segment are then fed to an SVM classifier. On the cleaned CC-CCII CT dataset, this patch-based sparse-coding pipeline achieved 89% accuracy for binary NCP-vs-other classification and 82.6% accuracy for the three-class problem, matching or slightly exceeding a benchmark ResNet3D-34 deep learning baseline on the same data. The authors emphasize that because dictionary atoms live in the same domain as the input patches, the resulting classifier offers a more interpretable link between learned features and diagnostic categories than an end-to-end deep network directly echoing the discriminative-dictionary theme seen in Dong et al. and Tong et al., but here applied to whole-scan disease classification rather than pixel-wise segmentation.

Chapter 3

Methodology

Figure 3.1 gives a visual overview of the full framework presented in this study.

Figure 3.1: Overall Methodology Diagram.

3.1 Data Acquisition

The quality and consistency of the input data has a direct impact on how well the dictionary learning stage can later separate abnormalities from normal lung, so a fair amount of the early project time went into data acquisition and preparation before any model code was written. This section walks through the datasets explored and chosen, how the raw CT volumes were loaded and experimented on, the different lung segmentation strategies that were tried, and how the selected lung volumes were sampled into 3D patches that feed the Dictionary Learning models.

3.1.1 Dataset Selection

Multi-abnormality classification and segmentation for 3D chest CT images required a dataset that included multiple CT abnormalities and their corresponding voxel-level annotations. Only three datasets fulfilled the former requirement:

1. CT-RATE[25]
2. RadGenome-Chest CT[**?**]
3. Rad-ChestCT[**?**]

Of these, only CT-RATE and RadGenome-Chest CT were publicly available, while Rad-ChestCT only had an initial release of 3,630 chest CT scans (approximately 10% of the dataset) and the files were restricted to the public. Rad-ChestCT was also used as the external validation set for training the CT-CLIP model, which was trained on the CT-RATE dataset. RadGenome-Chest CT extends the CT-RATE dataset with with 197 category organ-level masks and 665K grounded text reports for vision-language AI models. However, neither of these provides the necessary spatially localized annotations to train a 3D lung segmentation model. Fortunately, ReXGroundingCT[58] was released very close to the start of this project, and it provides per-finding voxel-level segmentation masks for a subset of the CT-RATE dataset.

CT-RATE Dataset

CT-RATE [25] is a large-scale 3D chest CT dataset in NifTI format, hosted on Hugging Face () that provides volume-level multi-label annotations and corresponding textual radiology reports across 18 distinct abnormality categories. The dataset comprises 25,692 non-contrast 3D chest CT scans from 21,304 unique male and female patients expanded to 50,188 total volumes via different reconstructions. It is split into a training cohort of 47,149 volumes from 20,000 patients and a validation cohort of 3,039 volumes from 1,304 patients. Table 3.1 shows the 18 abnormality categories in CT-RATE.

Medical material	Lung nodule
Arterial wall calcification	Lung opacity
Cardiomegaly	Pulmonary fibrotic sequela
Pericardial effusion	Pleural effusion
Coronary artery wall calcification	Mosaic attenuation pattern
Hiatal hernia	Peribronchial thickening
Lymphadenopathy	Consolidation
Emphysema	Bronchiectasis
Atelectasis	Interlobular septal thickening

Table 3.1: List of abnormalities in the CT-RATE dataset.

CT-RATE captures substantial heterogeneity in CT scanners, acquisition protocols, and reconstruction techniques. It comprises scans from four primary scanner manufacturers:

1. Siemens Healthineers
2. SIEMENS
3. Philips Healthcare
4. Philips-Neusoft Medical Systems

ReXGrounding-CT Dataset

ReXGroundingCT [58], is the first publicly available dataset linking voxel-level 3D segmentation masks to free-text findings in chest CT scans in NifTI fromat, hosted on Hugging Face (). It includes 3,142 non-contrast chest CT studies derived from the CT-RATE dataset organised as a train set of 2,992 scans, a valid set of 50 scans and a held-out private test set of 100 studies. This test set is hosted on RexRank [59], a public leaderboard and challenge for assessing AI-powered radiology report generation. ReXGroundingCT enables grounding for 14 abnormality categories with 79% of the annotated findings represent focal abnormalities such as nodules or localized consolidations, while 21% represent diffuse or infiltrative patterns. Table 3.2 shows the list of abnormalities in the ReXGroundingCT dataset, divided into typically focal and typically non-focal categories.

A segmentation mask is 4-Dimensional ($F \times H \times W \times D$) where F is the number of findings. Scans feature HxW resolutions primarily at 512x512 pixels with slice counts spanning 104 to 1,005 slices. Background Pixels are represented by a value of 0. For

Table 3.2: List of Abnormalities in the ReXGroundingCT Dataset

Typically Focal	**Typically Non-Focal**
Pulmonary nodules and masses	Emphysema (including Centrilobular,
Consolidation / atelectasis	Paraseptal, Bullous)
Ground-glass opacities	Bronchiectasis
Pleural effusion / thickening	Bronchial wall thickening
Linear opacities (including subsegmental	Septal thickening (including Interlobular,
atelectasis, scarring, fibrosis)	Reticulation)
Honeycombing	Micronodules (including Centrilobular,
Pneumothorax	Tree-in-bud, Perilymphatic)
Other focal findings	Other non-focal findings

any given channel along the F dimension, each unique non-zero integer represents a distinct entity instance belonging to that finding.

3.1.2 Target Abnormality Selection

The target abnormality classes were selected based on several factors:

- Expert Radiologist Recommendation
- Sufficient Representation in the Dataset
- Clinical Significance

To obtain an expert recommendation from a radiologist, a consultation session was held with a Consultant Radiologist at the Teaching Hospital, Karapitiya to discuss the available findings in the ReXGroundingCT datasets. Figure 3.2 shows a group photo taken after the consultation session.

Figure 3.2: First consultation session with the Consultant Radiologist

The radiologist was shown the available finding categories in ReXGroundingCT and asked which abnormalities would be most valuable to detect automatically, and which were common enough in practice to justify the modelling effort. The radiologist

highlighted the following findings as having a well-established radiological basis for early intervention, making them good candidates for a proof-of-concept CAD system:

- Ground-glass opacity
- Pulmonary nodule
- Consolidation
- Mosaic attenuation
- Bronchiectasis
- Emphysema

The radiologist provided valuable insights into the clinical significance of each abnormality, their typical appearance in CT scans, and the challenges associated with their detection. For the initial phase of our study, we decided to narrow down the target abnormalities further, as the radiologist's recommendations were quite broad.

However, we also observed discrepancies between the abnormality classes in the CT-RATE and ReXGrounding-CT datasets. Firstly, CT-RATE does not have a class for GGO explicitly. Instead, it has a broader class called "Opacities", which contains various generalized density increases, non-specific haziness, and explicit GGOs grouped together under this broader class. This is offset by the fact that ReXGrounding-CT treats GGO as an independent, dedicated focal category. Secondly, ReXGrounding-CT groups Atelectasis and Consolidation as one finding.

This could be because they share nearly identical visual appearances and overlapping imaging features on chest CT scans [60]. Both conditions cause increased density and whitening of the lung parenchyma that obscures underlying blood vessels, and frequently present with visible air bronchograms within the affected area. This is further confirmed by the ReXGrounding-CT free-text reports, which describe many findings as "consolidation or atelectasis".

The categorization pipeline and schema were constructed via LLM extraction and verified by board-certified radiologists [58], which suggests that the grouping was a deliberate decision made by the authors. However, another study [61] provides a comprehensive hierarchical taxonomy, with 404 representative abnormal findings across all body regions. Table 3.3 shows the abnormalities listed under the "Lung" category.

This study separates Atelectasis and Consolidation into two distinct categories, which is a more granular approach than ReXGrounding-CT. Through consultation with the radiologist, it was decided that Atelectasis and Consolidation should be distinct categories as they represent different pathological processes as noted in 2.1.2. Because there are practical constraints in dividing the already-grouped Atelectasis/-Consolidation category in ReXGrounding-CT, we decided to remove them from the initial phase of the study. To further reduce the target abnormalities, we considered the representation of each class in our dataset which is discussed in section 3.1.3. We finally decided on 2 target abnormalities for the initial phase of the study:

1. GGO
2. Nodules/masses

Table 3.3: Pulmonary abnormality taxonomy in OmniAbnorm-CT.

Category	Abnormalities
Pulmonary parenchyma	Atelectasis; Incomplete lung expansion; Pulmonary consolidation; Pulmonary ground-glass opacities; Pulmonary emphysema; Pulmonary solitary nodule or mass; Pulmonary diffusely distributed multiple nodules; Pulmonary parenchymal fibrosis; Pulmonary thin-walled cavitation; Pulmonary thick-walled cavities; Pulmonary cystic mass.
Bronchi	Bronchiectasis; Bronchial wall thickening, stenosis, or occlusion; Bronchial foreign body.
Lung interstitial	Lung interstitial fibrosis and thickening; Honeycomb lung.
Pleura	Pleural thickening; Pleural effusion; Pneumothorax; Pleural calcification; Pleural soft tissue mass.

3.1.3 Exploratory Data Analysis

Based on the radiologists feedback, we performed an exploratory data analysis (EDA) and cross-checked against how well each candidate class was actually represented in the annotated data. Table 3.4 shows the distribution of the 14 finding categories in ReXGroundingCT.

Based on the number of findings, it was clear that GGO, Pulmonary Nodules/-masses and Atelectasis/Consolidation were the most well-represented categories in the dataset, with 1,527, 1,778 and 1,384 findings respectively. Since we decided to remove the grouped Atelectasis/Consolidation category, we were left with GGO and Pulmonary Nodules/masses as the two main target abnormalities.

Beyond how many findings each category contributed, it was also important to understand how large a typical finding actually is, since this has a direct impact on how patches would later need to be sampled around each lesion. For every GGO and nodule/mass annotation in the dataset, the number of foreground voxels enclosed by its ground-truth mask was measured, giving one voxel count per annotation. Table 3.5 summarises the resulting distributions.

The two categories turned out to have very different, heavily right-skewed size profiles. The median GGO annotation covers 19,682 voxels, roughly 47 times larger than the median nodule/mass annotation at just 417 voxels. Both distributions also have a long tail of much larger outliers, with the largest single GGO annotation reaching 3,442,007 voxels and the largest nodule/mass annotation reaching 1,872,872 voxels, which is why the histograms in Figures 3.3 and 3.4 are plotted on a $\log_{10}$(voxel count) axis rather than a linear one.

Figure 3.3: Log-scale voxel-count distribution of GGO (category 2c) annotations.

Figure 3.4: Log-scale voxel-count distribution of Nodule/mass (category 2d) annotations.

Table 3.4: Distribution of Findings and Scans Across ReX Categories

Category	**# Scans**	**# Findings**	**Description**
1a	213	237	Bronchial wall thickening
1b	249	285	Bronchiectasis
1c	421	449	Emphysema (including centrilobular, paraseptal, bullous)
1d	188	198	Septal thickening (including interlobular septal thickening, reticulation)
1e	246	321	Micronodules (including centrilobular, tree-in-bud, perilymphatic)
1f	145	153	Other non-focal abnormalities
2a	815	1135	Linear (including subsegmental atelectasis, scarring, fibrosis)
2b	924	1384	Atelectasis, consolidation
2c	1144	1527	Groundglass opacity
2d	1333	1778	Pulmonary nodules/masses
2e	186	241	Pleural effusion or thickening
2f	15	16	Honeycombing
2g	15	18	Pneumothorax
2h	58	60	Other focal abnormalities

Table 3.5: Voxel-count statistics of GGO (2c) and Nodule/mass (2d) annotations.

Category	**# Ann.**	**# Scans**	**Mean**	**Median**	**P25**	**P75**
GGO (2c)	1,543	1,161	113,871	19,682	4,818	83,380
Nodule/mass (2d)	1,801	1,349	10,196	417	194	1,167

This size disparity between the two target abnormalities turned out to be one of the more practically important findings of the EDA stage, since a fixed patch-level extraction rule that works for one category does not necessarily transfer to the other. A GGO region is typically large enough that a $16 \times 16 \times 16$ patch sampled from anywhere near its centre is likely to be mostly lesion tissue, whereas a nodule is frequently small enough that a patch covering its bounding box will contain mostly surrounding normal parenchyma. This is the underlying reason for the category-specific lesion-voxel-fraction thresholds used later during patch extraction (Section 3.2.4) – a uniform 50% threshold, while reasonable for GGO, would have discarded the majority of genuine nodule patches, so a lower 10% threshold was used for the nodule/mass category instead.

3.1.4 CT Volume and Mask Loading

Each CT scan is stored as a 3D NIfTI volume of dimensions $512 \times 512 \times N_z$ with voxel spacing $\Delta_x, \Delta_y, \Delta_z$ mm and 16-bit signed integer intensities representing Hounsfield Units (HU). The corresponding segmentation mask is stored as an 8-bit unsigned integer 4D NIfTI volume with matching spatial dimensions plus an additional singleton channel dimension containing integer label values.

Each NifTI file contains a header with metadata describing the volume's attributes. Table 3.6 lists the relevant header fields used in this study.

Table 3.6: NIfTI header attributes for the CT volumes and corresponding segmentation masks.

Header Attribute	**CT Volume**	**Segmentation Mask**
Dimensions	[3, 512, 512, N_z, 1, 1, 1, 1]	[4, 1, 512, 512, N_z, 1, 1, 1]
Data type	`int16`	`uint8`
Bits per voxel	16	8
Voxel spacing	$[1, \Delta_x, \Delta_y, \Delta_z, 0, 0, 0, 0]$	$[1, 1, 1, 1, 0, 1, 1, 1]$
Spatial units	Millimetres	Millimetres
Intensity scaling	(s, i)	(s, i)
Affine transformation	Scanner-space affine	Identity affine
Image origin	(t_x, t_y, t_z)	$(0, 0, 0)$

The CT volumes preserve the scanner-space affine transformation, which maps voxel indices to physical coordinates in millimetres. In general, the affine matrix can be represented as:

$$\mathbf{A} = \begin{bmatrix} -\Delta_x & 0 & 0 & t_x \\ 0 & -\Delta_y & 0 & t_y \\ 0 & 0 & \Delta_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where Δ_x, Δ_y, and Δ_z denote the voxel spacing along the x-, y-, and z-axes, respectively, and (t_x, t_y, t_z) denotes the translation of the image origin in scanner coordinates. The negative scaling factors along the x- and y-axes indicate that these axes are stored in the opposite direction relative to the scanner coordinate system, while the z-axis retains its positive orientation, which is common for DICOM-derived CT scans.

Although each segmentation mask corresponds exactly with its associated CT scan, the mask NIfTI headers do not preserve the original spatial metadata. Specifically, the masks are stored with unit voxel spacing ($1 \times 1 \times 1$ mm), and an identity affine transformation, whereas the CT volumes retain their scanner-space affine and true voxel spacing. During preprocessing, the mask metadata is updated to match the corresponding CT volume's metadata.

Volumes were loaded using the nibabel Python library and converted into float32 HU arrays, with each scan represented as:

$$I(x, y, z) \in \mathbb{R}^{H \times W \times D}$$

where H, W and D denote the height, width and depth of the volume.

3.2 Data Preprocessing

Figure 3.5 summarises the overall data pre-processing pipeline used in this study.

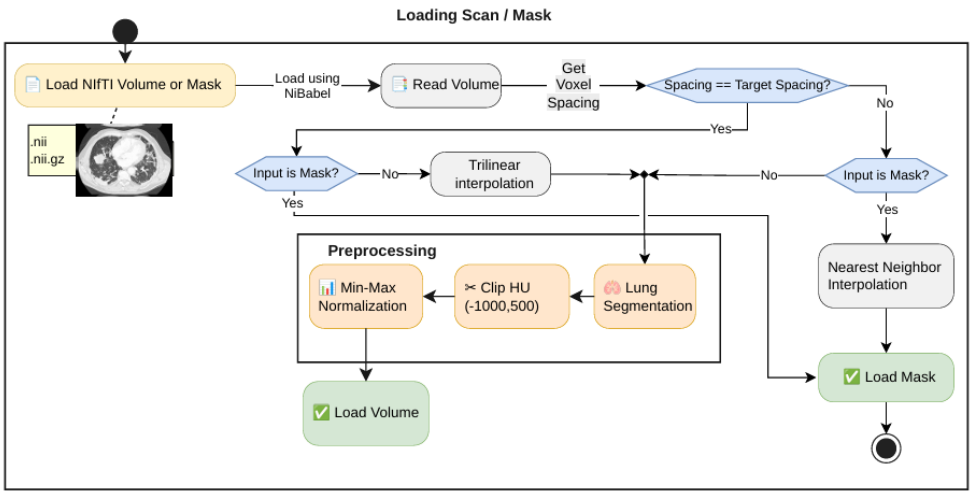

Figure 3.5: Shared loading pipeline for CT volumes and finding masks

3.2.1 Resampling

Due to the variation in voxel spacing across the CT-RATE dataset, all volumes had to be resampled to a common isotropic voxel spacing to correct for variable scanner resolutions. We picked the same resampling target spacing as the CT-CLIP model, since it was trained on the volumes from the CT-RATE dataset [25]. The target spacing was set to $0.75 \times 0.75 \times 0.75$ mm, in the x, y and z directions respectively.

An in-plane (X-Y) resolution of 0.75 mm is small enough to capture subtle pulmonary nodules without losing data to excessive downsampling. Furthermore, chest CTs naturally have much higher in-plane resolution than axial slice thickness (Z). Setting a 1.5 mm Z-spacing respects this clinical reality while compressing file sizes by 50% compared to isotropic 0.75 mm 3D grids.

The resampling was performed using the `scipy.ndimage.zoom` function with linear interpolation for the CT volumes and nearest-neighbour interpolation for the segmentation masks. The zoom factors were calculated as:

Figure 3.6: Distribution of voxel spacings across the dataset

$$\text{zoom}_x = \frac{\Delta_x}{0.75}, \quad \text{zoom}_y = \frac{\Delta_y}{0.75}, \quad \text{zoom}_z = \frac{\Delta_z}{1.5} \tag{3.1}$$

This was further confirmed by analysing the distribution of voxel spacings across the dataset. Figure 3.6 shows this distribution along each axis.

After resampling, all volumes and masks were resized to a fixed matrix size of $480 \times 480 \times 240$ voxels across the coronal, sagittal and axial axes respectively, using center-cropping for oversized volumes or padding for undersized volumes. This yields a field of view of $360 \times 360 \times 360$ mm, which spans the physical volume of a human thorax, ensuring critical boundaries are not cut off during cropping. The dictionary learning model also relies on non-overlapping $12 \times 12 \times 12$ voxel patches, so A matrix of $480 \times 480 \times 240$ voxels is divisible by 12 along each axis, which yields an exact patch grid of $40 \times 40 \times 20$ (384,000) patches per volume.

3.2.2 Lung Segmentation

Passing the whole volume to the model would include information on bones, soft tissue and background air which adds computational overhead without contributing anything useful to the models learning process, or worse, risks letting the model pick up spurious patterns unrelated to lung pathology. Therefore, before any patches could be extracted, the raw volumes had to be reduced to just the lung region. We explored three different approaches to isolating the lungs were tried over the course of the project.

Classical Image Processing

The first attempt used a fairly standard HU-threshold pipeline, since lung tissue is normally well separated from surrounding structures in intensity. Each volume was first thresholded to keep only voxels between -900 and -200 HU, the range typically associated with aerated lung parenchyma:

$$M_{thresh}(x, y, z) = \begin{cases} 1, & -900 \leq I(x, y, z) \leq -200 \\ 0, & \text{otherwise} \end{cases} \tag{3.2}$$

This crude threshold mask is noisy on its own, so it was cleaned up with a short sequence of morphological steps applied slice-by-slice and volume-wise using `scipy.ndimage` and `skimage.segmentation`. Border-connected regions were removed on each axial slice with `clear_border` (mainly to drop the surrounding air and imaging-table artefacts that also fall inside the HU window), after which only the two largest connected components were kept under the assumption that these correspond to the left and right lung:

A 3D binary closing (a $3 \times 3 \times 3$ connectivity structure, 2 iterations) was then used to smooth the boundary and reconnect thin gaps, followed by `binary_fill_holes` to plug any internal cavities left inside the mask – these two steps are exactly what a lung mask needs, since vessels, bronchi and small nodules would otherwise show up as holes in an otherwise solid lung region.

On visibly normal scans, this five-stage pipeline produced reasonable lung masks when checked manually. However, Consolidation and areas of lung opacity raise the local tissue density towards that of soft tissue, so the affected voxels are not registered as lung parenchyma under the initial -900 to -200 HU window. Since the lung segmentation fails at the very first thresholding step, we were motivated to move away from purely intensity-based segmentation and towards a learned segmentation model.

U-Net Model

The method that was ultimately adopted is the LungMask library, which uses the U-net (R231) model which is pretrained on diverse CT scan data for general lung mask generation. Unlike the classical pipeline, LungMask is a trained segmentation network built specifically for whole-lung extraction, and it proved noticeably more stable when abnormal tissue was present. Consolidation and GGO regions inside the lung boundary appeared to retain on manual inspection.

Given a CT volume, LungMask returns a label map with three values: 0 for background, 1 for the left lung and 2 for the right lung. This was converted into a binary mask and applied to the original volume.

Voxels falling outside the mask were reset to a fixed background value of -1000 HU (approximating air), after which the volume was clipped to the range $[-1000, 500]$ HU and linearly rescaled to $[0, 1]$. This combination of segmentation, HU clipping and normalisation was wrapped into a single preprocessing routine that also resamples each volume to an isotropic 1.5 mm spacing beforehand, therefore that every scan reaching the patch extraction stage has a consistent voxel grid regardless of its original acquisition parameters.

3.2.3 Normalization

3.2.4 Patch Extraction

Feeding entire volumes directly into the dictionary learning stage is not practical as a single scan can contain tens of millions of voxels. Therefore, the final step was to break each volume down into smaller 3D patches. Patch-based sampling provides the following benefits:

- Allows the model to focus on local structural patterns rather than whole-organ context.
- Reduces memory requirements by limiting the number of voxels processed at once.
- Since the proposed dictionary learning models are linear, the abnormal patches can be mapped back to the original volume.

To determine the dimensions of the patches, we considered the size distribution of the target abnormalities. The median GGO annotation covers 19,682 voxels, roughly 47 times larger than the median nodule/mass annotation at just 417 voxels. Since abnormalities are irregularly shaped, a maximum of 50% of the voxels in a patch were allowed to be non-lesion. A similar approach was used for healthy tissue, a maximum

Figure 3.7: 3D visualization of patch sampling from a lung volume.

Patch Size	# voxels	50% of Voxels
$32 \times 32 \times 32$	32,768	16,384
$16 \times 16 \times 16$	4,096	2,048
$12 \times 12 \times 12$	1,728	864
$8 \times 8 \times 8$	512	256

Table 3.7: Comparison of candidate patch sizes.

of 50% of the voxels in a patch were allowed to be background since HU values were set to -1000 outside the lung parenchyma. Figure **??** shows a visualization of a patch-sampled lung in 3D.

At 50% of the median nodule size, we considered multiple patch sizes that would include a sufficient number of patches from the nodule/mass category. Table 3.7 shows the comparison of two patch sizes that were considered.

Based on the distribution in figure **??**, selecting the $16 \times 16 \times 16$ patch size covers more than 25% of the patches in the nodule/mass category, while the $12 \times 12 \times 12$ patch size covers less than 10% of the patches in the nodule/mass category.

Each patch is a fixed $12 \times 12 \times 12$ voxel cube, which at the resampled spacing corresponds to roughly a $36\,\mathrm{mm}^3$ region of lung tissue large enough to capture a typical small nodule or a local patch of opacity, without dragging in unrelated anatomy. Patches were sampled differently depending on whether they were meant to represent normal tissue or a target abnormality:

- **Normal patches** were drawn on a non-overlapping grid across the entire lung volume, with a stride equal to the patch size. A patch was discarded if more than half of its voxels were near-zero after masking, since that generally indicates the patch sits mostly outside the lung region rather than within actual tissue.
- **Abnormal patches** were sampled only from scans carrying a ground-truth finding mask. Since a single scan can have several individual findings that belong to the same target category, the relevant finding channels of the 4D mask were first OR-combined into one binary lesion mask per category present in that scan. A tight axis-aligned bounding box was then computed around the foreground of that lesion mask, and the patch window was slid across the box with a stride of 4 voxels in each direction, making it markedly denser than the normal-patch grid, since the region of interest here is already small and every overlapping view of it is potentially useful. Each candidate patch was only kept if the fraction of lesion-labelled voxels within its corresponding mask region reached a threshold of 50%.

Figure 3.8 lays this out visually.

Each extracted patch is subsequently flattened into a vector representation before being handed to the dictionary learning algorithms:

$$x_i \in \mathbb{R}^n, \quad n = p^3 = 12^3 = 1,728 \tag{3.3}$$

Patches extracted this way were saved per scan and per class as compressed archives containing the flattened feature matrix, class labels, the originating scan

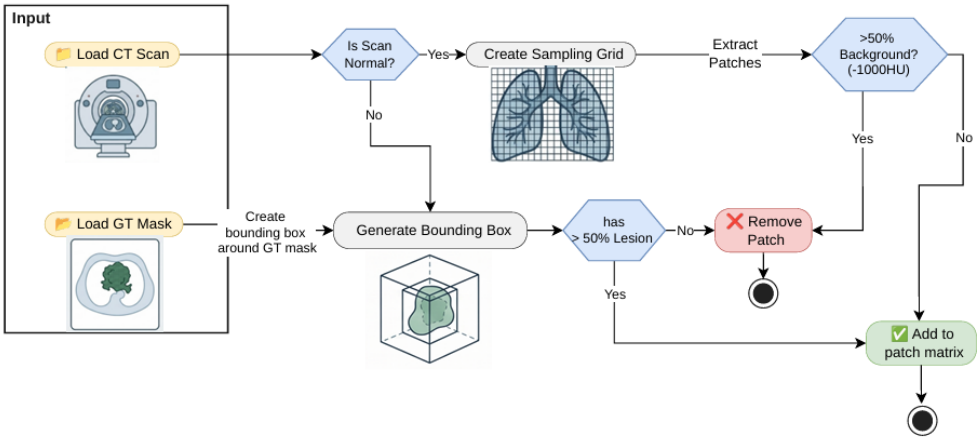

Figure 3.8: Patch extraction workflow, showing the normal-scan sampling grid and the lesion bounding-box branch for annotated scans, each filtered by its own background/lesion-fraction threshold before patches are added to the patch matrix.

identifiers and the voxel coordinates of each patch, and were later concatenated across classes to build the full training matrix used for the dictionary learning algorithms.

3.3 Model Training

Once class-wise patches had been extracted and flattened into vectors, the next stage was to learn a dictionary that could represent these patches as a sparse combination of a small number of atoms. This section describes the dictionary learning algorithms that were implemented, the training pipeline used to run them, and the hyperparameters that control the process. All of the dictionary learning code lives in a separate open-source PyPI package built specifically for this project https://pypi.org/project/reppi/. The code can be found at: https://github.com/ckekula/reppi.

Four dictionary learning algorithms were implemented in `reppi` and are selectable from the training entry point via an `--algorithm` flag: `ksvd`, `lcksvd`, `fddl`, and `frozen`. All four take the same flattened patch matrix $X \in \mathbb{R}^{n \times N}$ (where $n = 1728$ is the flattened patch length and N is the number of patches) and produce a dictionary D that patches can later be sparse-coded against.

3.3.1 KSVD

K-SVD is the standard dictionary learning algorithm of Aharon et al. [44]. It alternates between sparse-coding all patches against the current dictionary and updating each atom one at a time to better fit the patches that use it. It is unsupervised, meaning it is not told which class a patch belongs to. It is trained on all patches (normal, GGO and nodule) together and forms the base building block that the other three algorithms are built on top of.

Before an atom can be updated, K-SVD first needs to know which atoms are being used to reconstruct each patch. This step, sparse coding, is performed the same way inside every K-SVD-based algorithm in this project (K-SVD itself, LC-KSVD, and Frozen Dictionary Learning), using **Batch Orthogonal Matching Pursuit (Batch-OMP)**, following Elad et al. [62].

OMP is a greedy algorithm. Given a patch x and a dictionary D whose columns (atoms) have unit norm, it picks the atom most correlated with the current residual, adds it to the active set, solves a least squares problem over the active atoms, and repeats until either a fixed number of atoms have been chosen or the residual is small enough. In this project, the number of atoms is capped at a fixed sparsity level $T = 10$ for every algorithm, meaning a patch is represented using at most 10 dictionary atoms out of several thousand.

Batch-OMP is a reformulation of this same idea that avoids repeating expensive matrix work when many patches are encoded against the same dictionary at once, which is the normal case here since every K-SVD iteration re-encodes the whole training set. Instead of working with D and each patch directly, it precomputes the Gram matrix $G = D^\top D$ and the correlation matrix $D^\top X$ once for the whole batch of patches, and reuses these to solve for every patch's sparse code. This is different from the FISTA-based, L1-regularised sparse coding described later in Section 3.4.1, which is used to generate sparse codes for the downstream classifier once a dictionary has already been learned and frozen. Batch-OMP, by contrast, is used only during dictionary training itself, where a fixed maximum number of active atoms per patch (rather than an L1 penalty) is what the K-SVD algorithm expects.

3.3.2 LC-KSVD

LC-KSVD (Label Consistent K-SVD), following Jiang et al. [45], extends K-SVD so that the dictionary also becomes discriminative. It adds a label-consistency term that pushes atoms associated with one class to produce similar sparse codes for patches of that class, and, in the LC-KSVD2 variant used here, jointly learns a linear classifier alongside the dictionary. Unlike K-SVD, LC-KSVD is supervised: it needs the class label of every patch during training.

3.3.3 Frozen Dictionary Learning

Frozen Dictionary Learning, implemented as `IncrementalFrozenDictionary` and following Carroll et al. [47], is the default algorithm used by `main.py` and is the primary method used in this project. It learns a dictionary in stages rather than all at once: a base dictionary is learned first on normal patches only, and its atoms are then frozen (held fixed) while a residual dictionary is learned for each abnormality class in turn. Section 3.3.5 describes this in more detail, since it is the method actually used to produce the dictionaries evaluated in this project.

K-SVD is the shared core of every algorithm used in this project. Given the patch matrix X, a dictionary D of K atoms and a sparsity target T, K-SVD repeats the following steps for a fixed number of iterations (set to 10 in this project):

1. **Sparse coding.** Every patch in X is encoded against the current dictionary D using Batch-OMP, giving a sparse code matrix Γ such that $X \approx D\Gamma$.

2. **Atom update.** Each atom d_j (column j of D) is updated one at a time. For atom j, the patches that currently use it are found, and the part of their reconstruction error that is not explained by any other atom is collected into a residual matrix. A rank-1 approximation of this residual matrix (computed with a truncated SVD in this project's configuration, rather than a full SVD, for

speed) gives an updated atom d_j together with updated sparse coefficients for the patches that use it. Because each atom is updated using the latest version of every other atom, this tends to converge faster than updating all atoms at once.

3. **Dead atom handling.** If an atom is not being used by any patch, or if it has become too similar to another atom (a mutual coherence above a threshold of 0.99), it is replaced with the patch that is currently worst reconstructed by the dictionary. This keeps every atom useful and stops the dictionary from wasting atoms on near-duplicate patterns.

K-SVD in this project also supports a set of **frozen atoms**: a block of atoms that can be included in the dictionary during sparse coding and error computation, but that are never touched by the atom update or dead-atom replacement steps. This is what makes Frozen Dictionary Learning possible, since it lets a new set of atoms be trained on top of an already-trained, unchanging set of atoms from an earlier stage.

3.3.4 FDDL

FDDL (Fisher Discrimination Dictionary Learning), following Yang et al. [46], builds a dictionary made of one sub-dictionary per class, so that each sub-dictionary reconstructs its own class well while poorly reconstructing the other classes. It further pushes the sparse codes of a class to be similar to each other and different from the codes of other classes, borrowing the idea of within-class and between-class scatter from Fisher discriminant analysis.

3.3.5 Frozen Dictionary Learning (Primary Method)

Frozen Dictionary Learning is the algorithm used by default in this project (`main.py` with no `--algorithm` flag, or `--algorithm frozen`), and is the method used to produce the dictionaries analysed later in this chapter. The motivation for this approach is that normal lung tissue makes up the large majority of voxels in any CT scan, while GGO and nodule patches are comparatively rare. Training a single K-SVD dictionary jointly on all three classes risks having the dictionary's atoms dominated by normal-tissue patterns, since a plain K-SVD run has no notion of class balance. Frozen Dictionary Learning avoids this by learning the normal-tissue representation first, then explicitly reserving new atoms for each abnormality on top of it, so the abnormality classes are guaranteed a dedicated part of the dictionary rather than having to compete for space with normal patches.

The training pipeline for this algorithm has two stages:

1. **Base stage.** A K-SVD dictionary is trained using only the normal patches in X (found by masking X against the label vector H for the normal class). This produces the base dictionary D_{base}, sized to $N\_FEATURES \times 4 = 6912$ atoms in this project's configuration.

2. **Residual stages.** For each abnormality class, in the fixed order `["2c", "2d"]` (GGO, then nodule), a new K-SVD instance is trained on that class's patches only, with the entire dictionary learned so far (the base dictionary, plus any earlier abnormality's residual dictionary) passed in as frozen atoms. K-SVD then

jointly sparse-codes every patch of this stage's class over the combined `[frozen atoms | new atoms]` dictionary at every iteration, but only ever updates the new atoms. Once this stage finishes, its new atoms are appended to the dictionary and frozen in turn for the next class. GGO is given $N\_FEATURES \times 2 = 3456$ new atoms and nodule is given $N\_FEATURES = 1728$ new atoms, reflecting how much less training data is available for nodules than for GGO.

The result is a single combined dictionary $D = [D_{\text{base}} \mid D_{2c} \mid D_{2d}]$ in which every atom's class of origin is known and recorded (as an atom index range per class), even though no atom was ever told this label during its own training. This bookkeeping is later used when inspecting which atoms are used for which class, as described in Section 3.4.3's discussion of atom activation.

3.3.6 Training Pipeline Walkthrough

The training entry point, `main.py`, and the pipeline it calls, `train.py`, run the following steps in order:

1. **Patch extraction (optional).** Unless `--skip-extraction` is passed, patch extraction (described in Section **??**) is run first to generate the per-class patch archives.

2. **Loading the unified patch matrix.** The per-class patch archives (normal, GGO, nodule) are loaded and concatenated along the patch axis into a single matrix X, together with the matching label vector H, scan identifiers, and patch coordinates.

3. **Shuffling.** If shuffling is enabled (the default), the patches in X, H, the scan identifiers and the coordinates are all permuted together using the same random permutation, seeded with a fixed random seed (42) for reproducibility. The shuffled matrix is also saved to disk so later runs can reuse the exact same patch order.

4. **Selecting and configuring the algorithm.** Based on the `--algorithm` argument, one of the four fitting functions (`_fit_frozen`, `_fit_lcksvd`, `_fit_fddl`, `_fit_ksvd`) is called with its corresponding hyperparameter configuration from `config.py`.

5. **Fitting.** The chosen algorithm is trained on X (and H, for the supervised algorithms). For the default frozen algorithm, this runs the base-then-residual pipeline described in Section 3.3.5. Training progress (RMSE per iteration) is logged, and a checkpoint is written to disk after every iteration so that a training run can be resumed if interrupted, rather than restarted from scratch.

6. **Saving.** Once training finishes, the fitted model object, the algorithm name, the class order, the hyperparameter configuration used, and metadata such as patch size and the HU window are packaged together and saved to disk as a single pickle file (for example, `unified_frozen.pkl`). This saved file is what the classification stage in Section 3.4 loads and encodes new patches against.

3.3.7 Hyperparameters

Table 3.8 summarises the main hyperparameters used for dictionary training in this project, taken directly from `config.py`. The patch size of $12 \times 12 \times 12$ voxels gives a flattened patch length of $N\_FEATURES = 12^3 = 1728$, which is used throughout to size the dictionaries (for example, the frozen algorithm's base dictionary is $N\_FEATURES \times 4$ atoms).

Table 3.8: Dictionary learning hyperparameters (from `config.py`)

Hyperparameter	**Value**	**Applies to**
Sparsity level T (`n_nonzero_coefs`)	10	All algorithms
K-SVD/base iterations (`n_iter`)	10	All algorithms
Mutual coherence threshold (`mu_thresh`)	0.99	K-SVD, Frozen
Base dictionary size	$N\_FEATURES \times 4 = 6912$	Frozen
GGO residual dictionary size	$N\_FEATURES \times 2 = 3456$	Frozen
Nodule residual dictionary size	$N\_FEATURES = 1728$	Frozen
Plain K-SVD dictionary size	$N\_FEATURES \times 7 = 12096$	K-SVD
LC-KSVD dictionary size	$N\_FEATURES \times 6 = 10368$	LC-KSVD
Label-consistency weight α	4.0	LC-KSVD
Classifier weight β (LC-KSVD2 only)	2.0	LC-KSVD
LC-KSVD outer iterations	10	LC-KSVD
LC-KSVD initialisation iterations	2	LC-KSVD
FDDL dictionary size	$N\_FEATURES \times 7 = 12096$	FDDL
Random seed	42	All algorithms

All four algorithms share the same sparsity level ($T = 10$) and number of iterations (10) so that comparisons between them are not skewed by one algorithm simply being given more training time or a more generous sparsity budget. Dictionary sizes differ between algorithms because each algorithm distributes its atoms differently: the frozen algorithm splits its atoms explicitly across the base and residual stages, while K-SVD, LC-KSVD and FDDL each learn a single dictionary sized as a multiple of `N_FEATURES`.

The dictionary produced by whichever algorithm is trained, whether the frozen dictionary's combined $D = [D_{\text{base}} \mid D_{2c} \mid D_{2d}]$ or the single jointly trained dictionary from LC-KSVD, FDDL or plain K-SVD, is saved to disk once training finishes and is not modified again. The next stage of the pipeline treats this dictionary as fixed: every patch, whether from the training set, the test set, or a brand new CT volume, is re-encoded against this same frozen dictionary using the FISTA-based sparse coding described in Section 3.4.1, and it is these sparse codes, not the raw patches, that the classifiers described in Section 3.4 are trained and evaluated on.

3.4 Classification

Sparse coding reduces each patch to a compact numerical representation Γ against the learned dictionary D, but Γ on its own is only a representation, not a prediction – nothing in the dictionary itself decides whether a given patch is normal, GGO or a nodule. A separate, standalone machine learning model is therefore trained on

top of the sparse codes to assign each patch a class label. This classification stage is implemented independently of the dictionary-learning code, and is run only once a dictionary has already been learned and frozen: the same classification pipeline is applied unchanged regardless of which dictionary-learning variant (K-SVD, frozen dictionary learning, or LC-KSVD) produced the sparse codes it is given.

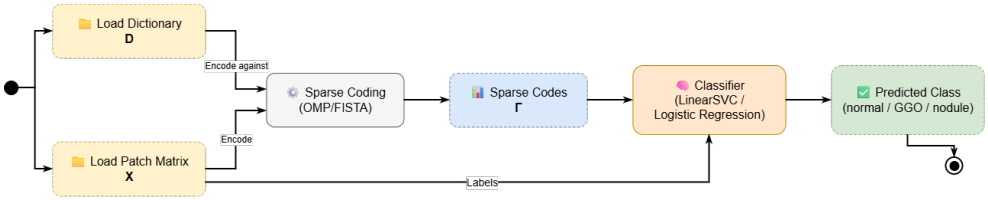

Figure 3.9: Overview of the classification pipeline: each patch is first converted into a sparse code Γ against the frozen dictionary D (Section 3.2.1), and the resulting codes are then used to train and query a classifier that predicts the patch-level class label (Section 3.2.2).

3.4.1 Sparse Coding

Before training the classifier, every patch in the training dataset is converted into a sparse representation using the learned dictionary D. This process represents each patch as a combination of dictionary atoms.

Given the patch matrix X and dictionary D, the sparse codes are obtained by solving an optimization problem that minimizes the reconstruction error while encouraging sparse solutions. This is achieved using an ℓ_1-regularised least-squares (LASSO-style) formulation,

$$\min_{\Gamma} \ \|X - D\Gamma\|_2^2 + \lambda\|\Gamma\|_1, \tag{3.4}$$

where $\lambda = 0.1$ controls the balance between reconstruction accuracy and the sparsity of the generated codes.

Before applying sparse coding, both the patch matrix X and the dictionary D are normalized to unit-norm columns. This prevents features with larger magnitudes from dominating the optimization process and ensures that all dictionary atoms contribute equally.

The optimization problem is solved using FISTA (Fast Iterative Shrinkage-Thresholding Algorithm). The project's own reppi solver core is used directly instead of the available sparse-coding wrapper because the wrapper only supports PyTorch tensors, while the patch extraction stage produces NumPy-based patch matrices.

The reconstruction error term is optimized using its gradient,

$$\nabla_\Gamma \|X - D\Gamma\|_2^2 = 2D^\top(D\Gamma - X), \tag{3.5}$$

while the sparsity constraint is handled using the soft-thresholding proximal operator,

$$\mathrm{prox}_{\lambda t}(v) = \mathrm{sign}(v)\max(|v| - \lambda t, 0), \tag{3.6}$$

which reduces small coefficient values towards zero and produces sparse representations.

FISTA uses a backtracking line search to automatically adjust the step size during optimization. The algorithm runs for a maximum of 500 iterations or stops earlier when the difference between consecutive updates becomes smaller than 10^{-8}.

Since each patch can be encoded independently, the training patches are processed in batches of 8192 samples instead of solving for the entire dataset at once. This reduces memory requirements because storing FISTA variables, including the current solution, momentum term, and gradients, for all patches simultaneously would require a large amount of memory. The computation is performed on the GPU when available, with a fallback to CPU execution otherwise.

After sparse coding is completed, the resulting sparse-code matrix Γ and the corresponding patch labels are stored on disk. This allows future training runs to reuse the generated sparse codes without repeating the complete sparse coding process.

3.4.2 Training

This stage uses the (Γ, label) pairs created in the previous stage to train a machine learning classifier. The transposed sparse coefficient matrix, $\Gamma^\top$, is used as the feature matrix, where each row contains the sparse code of one training patch. The corresponding label vector is used as the target. Using these features and labels, the classifier learns to predict the class of each image patch from its sparse representation.

Two classifiers are trained using the same (Γ, label) dataset. The first is a Linear Support Vector Classifier, which learns a linear decision boundary and is fully trained and tuned as the main classification model. The second is Logistic Regression, which predicts the probability of each class using the sparse code. Logistic Regression is included to compare its performance with the Linear SVC. The implementation and training of both classifiers are described in the following sections.

Linear Support Vector Classifier (LinearSVC)

The classifier used for classifying the sparse codes is scikit-learn's LinearSVC, combined with a MaxAbsScaler inside a single Pipeline.

The MaxAbsScaler scales each feature (i.e., each dictionary atom coefficient) by its maximum absolute value in the training data. It does not subtract the mean, which allows the zero values generated during sparse coding to remain unchanged. This is important because zero coefficients represent inactive dictionary atoms. In contrast, a mean-centering scaler such as StandardScaler would shift these zero values away from zero and reduce the sparsity information.

The regularisation parameter C of the LinearSVC classifier is selected using GridSearchCV, where the candidate values are $C \in \{0.1, 1.0, 10.0\}$. The selection process uses 5-fold cross-validation with macro-averaged F1-score as the evaluation metric instead of accuracy. This is because the three target classes (normal, GGO, and nodule) have an unequal number of training samples. The macro F1-score gives equal importance to each class by averaging the F1-score of all classes, preventing the classifier from being biased towards the majority class.

The LinearSVC model is configured with dual=False and a maximum iteration limit of 10,000 to ensure reliable convergence during training. The grid search is executed with n_jobs=2 and pre_dispatch=1 to control the computational resources used during hyperparameter tuning.

After the grid search identifies the optimal hyperparameter combination, the classifier is retrained on the complete training dataset using refit=True. The final trained model is then saved using joblib, allowing it to be reused during inference and evaluation without repeating the training process.

Logistic Regression

Logistic Regression

3.4.3 Inference

After training the classifier, its performance is evaluated using new patches that were not included during training. The testing process follows the same steps as the training pipeline, but it uses the separate test dataset. No additional training or parameter updates are performed during this stage. The classifier only predicts the class labels of the unseen patches, and the results are compared with the true labels to measure its performance. The same trained classifier can also be given a single new CT volume directly, with no patches or labels prepared for it in advance, in which case the pipeline extracts and classifies patches from that volume itself to produce a full per-voxel abnormality map.

Classification on the Test Set

Each test patch already has a sparse code and its corresponding label, generated using the same procedure as the training patches. At this stage, the trained machine learning models are only applied to the unseen test data for prediction.

The previously trained classifiers are loaded from storage and used to predict the class label of each test patch based on its sparse code. The same preprocessing steps used during training are applied before making predictions. No additional training, parameter tuning, or model updates are performed during this testing stage.

The predicted labels from each machine learning model are then compared with the true test labels using the evaluation procedure described in the next section. This allows the performance of different classifiers to be measured and compared on unseen data.

Whole-Volume Inference

Instead of using pre-extracted patches, the inference stage takes a single CT volume that the model has never seen before. The input volume is provided either as a path to a NIfTI file or through a scan identifier. No pre-generated patch archive or ground-truth mask is required for this process.

The objective of this stage is to generate a complete voxel-level abnormality map that can be visualised directly on the original CT scan. The inference pipeline reuses the main components introduced earlier, including preprocessing, patch extraction, sparse coding, and the trained classifier.

However, the process is slightly different because a new CT volume does not contain the pre-computed patch information available during training and testing. Therefore, patches must first be generated from the input volume, converted into sparse codes, and then classified to produce the final abnormality map.

Preprocessing The input volume is first taken through exactly the same preprocessing used at training time: it is resampled to the common 1.5 mm isotropic spacing, lung-segmented, HU-clipped and normalised to $[0, 1]$, so that a fresh scan reaching the classifier has been through the identical transformation as every training and test patch before it.

Dense patch extraction During training and testing, abnormal patches are extracted from regions around known lesions using ground-truth masks. However, a new CT volume does not contain a lesion mask, so the model cannot directly determine where abnormal patches should be extracted.

Instead, the entire preprocessed CT volume is scanned using a dense regular grid of $12 \times 12 \times 12$ voxel patches. A stride of 4 voxels is used, producing overlapping patches with three-fold overlap along each axis. This overlap improves the probability of detecting lesions even when they are located between two non-overlapping patch regions.

Similar to the normal patch filtering step used during training, patches containing more than half background or air voxels are removed before classification. The spatial origin coordinates of every remaining patch are stored together with the patch data. These coordinates are later used to map the predicted patch labels back to their original positions in the CT volume, allowing the generation of a voxel-level abnormality map.

Sparse coding and classification Each remaining patch is converted into a sparse code using the same dictionary D that was learned during the training stage. The same sparsity constraint is applied, allowing a maximum of 10 non-zero coefficients per patch. This ensures that all patches, whether they are from the training set, test set, or a new unseen CT volume, are converted into the same feature representation before classification.

The generated sparse codes are then provided to the trained machine learning classifiers. Each classifier uses these sparse features to predict the class label of every patch. The different classification models are applied independently, allowing their prediction results to be compared during evaluation.

The predicted labels and corresponding confidence scores (where supported by the classifier) are stored for each patch. These predictions are later used to generate the voxel-level abnormality map by mapping each patch prediction back to its original location in the CT volume.

Reassembling a per-voxel map A single patch's prediction only covers the small cube of voxels it was extracted from, and because patches overlap under the dense grid's stride, most voxels end up covered by several different patches, which will not always agree. Each patch's predicted label is therefore accumulated into a per-class vote count for every voxel it covers, and the final label at each voxel is decided by majority vote across all patches that covered it, with the same per-class decision score used to build a separate confidence "heat" volume wherever the winning label is an abnormal class. Since a classified patch is still accepted as a whole cube even when a substantial fraction of its own voxels are background (the same tolerance used for normal patches at training time), any voxel that is itself background in the preprocessed volume is forced back to the normal label regardless of what the patches

covering it predicted, so that an abnormal prediction cannot bleed out past the lung boundary into surrounding air or chest wall.

Reassembling a per-voxel map Each patch prediction represents only the small region of the CT volume covered by that patch. Since the dense grid produces overlapping patches, a single voxel may receive predictions from multiple patches, and these predictions may not always be the same.

To generate the final voxel-level abnormality map, the predictions from all patches covering each voxel are combined. For every voxel, the number of votes for each class is calculated, and the final class label is selected using a majority voting strategy. A separate confidence heatmap is also generated using the classifier decision scores, particularly for voxels predicted as abnormal.

Since each classified patch is considered as a complete cube, some patches may still include background voxels. To reduce false abnormal predictions outside the lung region, voxels that are identified as background in the preprocessed CT volume are forced to the normal class. This prevents abnormal predictions from spreading into non-lung regions such as surrounding air or the chest wall.

Statistical Analysis

The performance of the classifiers on the test dataset is evaluated using a common evaluating method. The same evaluation procedure is applied to all classifiers to ensure that the results are measured consistently regardless of the model used.

For each set of predictions and corresponding true labels, three main evaluation results are generated. The first is the overall accuracy, which represents the percentage of correctly classified test patches. The second is the classification report, which provides detailed performance metrics for each class individually. This report includes precision, recall, F1-score, and the number of samples belonging to each class (support). The third output is the confusion matrix, which shows the number of predicted samples compared with their actual class labels.

The per-class evaluation is important because the dataset does not contain an equal number of samples for all three target classes (normal, GGO, and nodule). A single accuracy value may appear high while hiding poor performance on classes with fewer samples. The classification report and confusion matrix provide a clearer understanding of the model performance by showing whether specific classes are incorrectly classified. For example, they can indicate whether nodule patches are more frequently misclassified as normal tissue or as GGO regions.

In addition to classifier performance evaluation, the quality of the sparse-code representation is also analysed. This analysis checks whether the learned dictionary and generated sparse codes behave as expected using measures such as sparsity, atom utilisation, and dictionary coherence.

Therefore, the evaluation process considers both the classification performance and the quality of the learned sparse representation, rather than relying only on classifier accuracy as an indication of model effectiveness.

Sparsity and atom utilisation The sparse codes generated using the fixed dictionary were analysed to evaluate the sparsity level and how effectively the dictionary

atoms were being used. The sparse-code matrix $\Gamma \in \mathbb{R}^{12096\times118705}$ contains one column for each training patch.

The results showed that most patches used the maximum allowed number of 10 non-zero coefficients. Out of the 118,705 training patches, 111,470 patches used exactly 10 dictionary atoms, resulting in an average of 9.39 active coefficients per patch.

However, some patches produced completely zero sparse codes. A total of 7,235 patches (slightly more than 6% of the dataset) resulted in all-zero representations, meaning that these patches contributed no useful information to the classifier. This represents a limitation of the sparse coding stage rather than a numerical or rounding error.

The utilisation of dictionary atoms was also highly unbalanced. One dictionary atom was activated in 14,209 patches, representing almost 12% of the dataset. This was significantly higher than the second most-used atom, which appeared in only 2.05% of patches. The remaining frequently used atoms among the top 15 were activated in approximately 1–2% of the dataset.

Due to the excessive usage of this single dominant atom, it was treated as an outlier and excluded from the subsequent atom co-occurrence analysis. This prevents the dominant atom from overwhelming the analysis and allows the relationships between the remaining dictionary atoms to be examined more clearly.

Atom co-occurrence and clustering The relationship between dictionary atoms was analysed by measuring how frequently pairs of atoms were activated together within the same patch. Only atoms that appeared at least 50 times in the training sparse codes were considered, and the most frequently used dominant outlier atom was excluded to prevent it from affecting the analysis.

The results showed that some atom pairs were activated together many times, with the strongest pairs having co-occurrence counts between 70 and 95 patches. Hierarchical clustering of atom activation patterns (Figure 3.10) showed that these relationships were not limited to a small number of individual pairs. Instead, atoms formed groups with similar activation behaviour, indicating that some atoms tend to be used together rather than independently.

A graph-based analysis was also performed by connecting atom pairs that were active together in at least 20 patches. Community detection on this graph identified a total of 32 atom groups. However, the group sizes were highly unbalanced. The five largest communities contained 149, 139, 128, 122, and 117 atoms, respectively, while most of the remaining communities contained only two to four atoms.

The co-occurrence heatmap (Figure 3.11) showed clear block patterns when atoms were arranged according to their clusters. The graph representation (Figure 3.12) also confirmed the presence of these groups by showing strongly connected regions of atoms.

Overall, the analysis demonstrates that dictionary atoms are not activated completely independently for each patch. Instead, groups of atoms tend to activate together, suggesting that these atoms capture related local image structures or patterns rather than representing isolated and unrelated features.

Aggregate atom contribution Summing the absolute sparse coefficient of every atom across a large batch of patches gives a sense of which atoms matter most once magnitude, not just activation frequency, is taken into account. Figure 3.13 plots

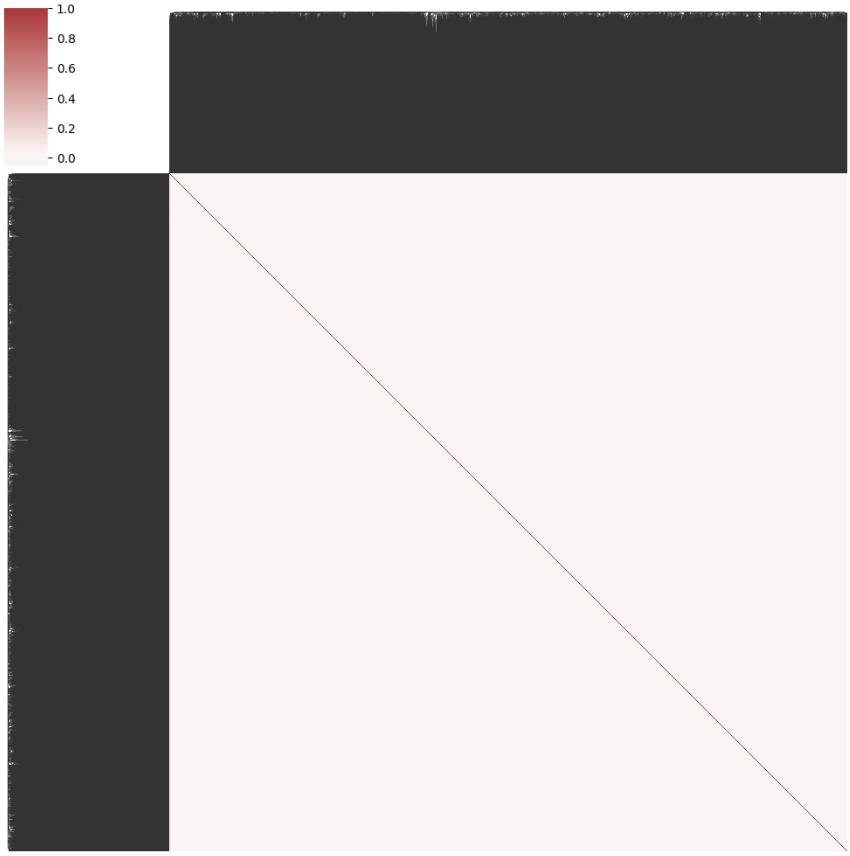

Figure 3.10: Hierarchical clustering of dictionary atoms by activation correlation across training patches (correlation-to-distance clustermap).

this aggregated contribution per atom across 117,386 patches: 28,671 atoms were active at least once, but the total is dominated by a small number of atoms with disproportionately large summed coefficients, standing well above the broad low-level background of atoms that contribute only marginally in aggregate. This is consistent with the uneven atom-utilisation pattern already observed – a small subset of atoms carries most of the representational weight, while the majority contribute a small, comparable amount each.

Reconstruction and visual inspection - As an additional qualitative evaluation, the quality of the sparse representation was visually analysed together with the numerical reconstruction error. The original patches were reconstructed from their sparse codes using $D\gamma$ and combined into a small volume for visual inspection (Figure 3.14).

The reconstructed volume showed recognizable lung structures and texture patterns rather than random noise. This indicates that the learned dictionary captures meaningful anatomical information from the CT images instead of learning arbitrary patterns.

The dictionary atoms were also visually inspected by selecting atoms from the normal, GGO, and nodule groups of the trained dictionary (Figure 3.15). However, the atoms from all three groups appeared visually similar, containing common image features such as local edges, intensity variations, small blobs, and texture patterns. No clear visual feature was observed that could distinguish a GGO or nodule atom

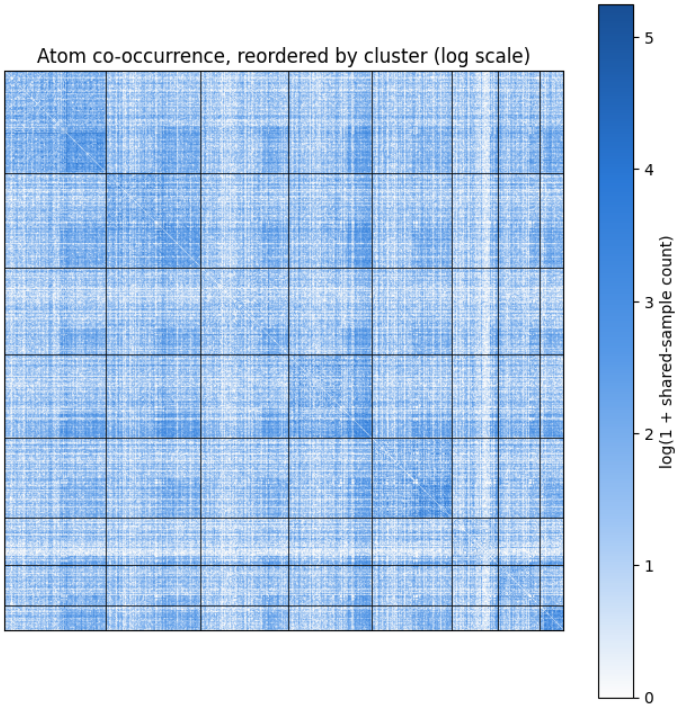

Figure 3.11: Atom co-occurrence matrix (log-scaled shared-patch counts), reordered so that atoms in the same community are adjacent.

from a normal tissue atom.

This behaviour is expected because the K-SVD dictionary learning process is primarily optimized for image reconstruction rather than class separation. Therefore, classification cannot rely on individual atom appearance alone. Instead, the downstream classifier uses the pattern of atom activations within the sparse code of each patch, as these activation patterns provide more useful information for distinguishing between the normal, GGO, and nodule classes.

Dictionary coherence. The dictionary coherence was analysed by measuring the similarity between every pair of normalized dictionary atoms. Instead of considering only the maximum coherence value, the complete distribution of pairwise similarities was examined.

The results showed that the dictionary does not contain a single pattern of similarity among its atoms. A large number of atom pairs had coherence values close to zero, indicating that these atoms are nearly independent and represent different image structures. This suggests that a significant portion of the dictionary contains unique and diverse information.

However, the coherence distribution also contained a group of atom pairs with moderate similarity, with coherence values around $|d_i^\top d_j| \approx 0.4$, along with a smaller number of highly correlated atom pairs. These strongly related atoms represent similar or redundant patterns within the dictionary.

Overall, the analysis indicates that most dictionary atoms are sufficiently distinct, allowing the dictionary to capture diverse image features. However, some groups of atoms contain similar information, which is consistent with the correlated atom clusters identified in the previous analysis.

Figure 3.12: Atom co-occurrence graph (spring layout), with the four largest detected communities coloured and all smaller communities grouped as "Other clusters".

3.5 Deep Learning Models

3.5.1 nnU-Net

3.5.2 nnU-Net Inference

nnU-Net was used as a deep learning segmentation model to detect the two target abnormalities identified with the radiologists: ground-glass opacity (GGO)/consolidation and pulmonary nodules. A separate pretrained binary segmentation model was used for each abnormality.

During inference, the trained nnU-Net models take a previously unseen CT volume as input and generate a complete voxel-level segmentation mask. Unlike the sparse-coding pipeline, which classifies individual image patches, nnU-Net directly predicts the abnormal regions for every voxel in the CT scan, producing a detailed segmentation of the target abnormalities.

Preprocessing During inference, each input CT volume is preprocessed using the same settings that were automatically determined by nnU-Net during the training and planning stage. These settings include the target voxel spacing, intensity normalization method, and patch size used for the `3d_fullres` configuration.

Using the same preprocessing ensures that every new CT scan is prepared in the same way as the training data. This allows the trained nnU-Net model to make accurate predictions without manually selecting preprocessing parameters. Unlike the sparse-coding pipeline, where the patch size and normalization method are manually defined, nnU-Net automatically applies the preprocessing settings learned during dataset planning.

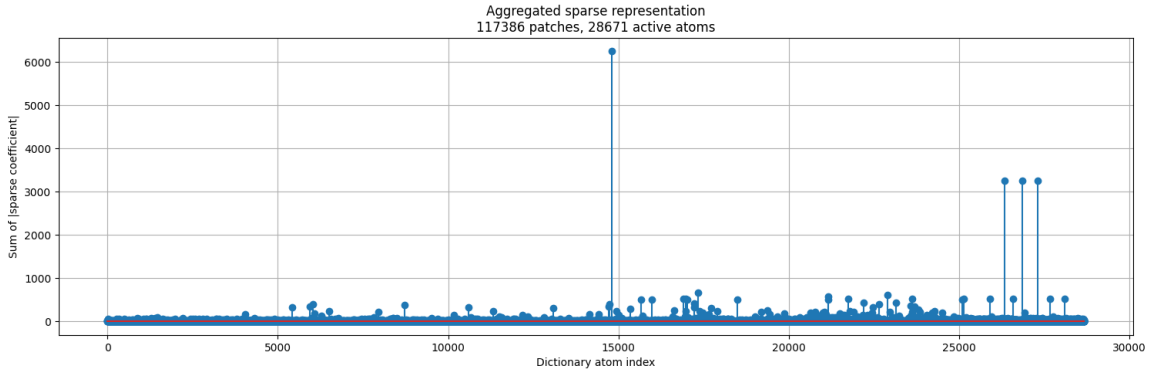

Figure 3.13: Aggregated sparse representation: summed absolute coefficient per dictionary atom across a large batch of training patches.

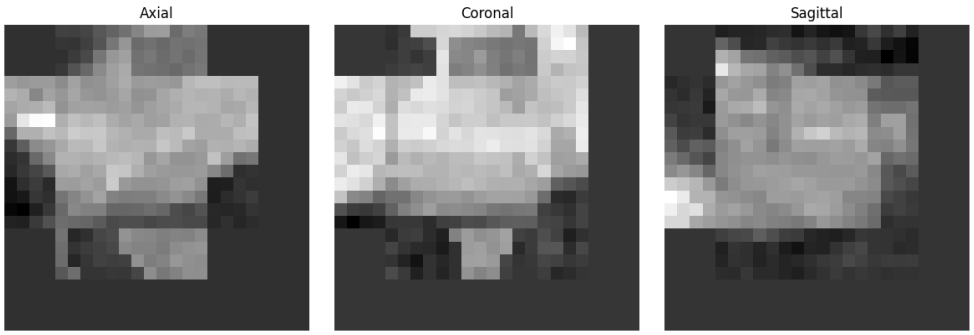

Figure 3.14: A small local volume reconstructed from its patches' sparse codes ($D\gamma$), shown in axial, coronal and sagittal view.

Inference During inference, the trained nnU-Net model is loaded and applied to previously unseen CT volumes. Each preprocessed CT scan is passed through the model to generate a voxel-level segmentation mask for the target abnormality.

The held-out test CT scans are then segmented.Predictions are generated by combining the available trained models through model ensembling. If multiple trained models are available, their predictions can be combined to improve the segmentation results. The predicted segmentation masks are then compared with the corresponding binary ground-truth masks used during training.

For each CT scan, the predicted and ground-truth masks are compared on a voxel-by-voxel basis by calculating the numbers of true positives, false positives, false negatives, and true negatives.

From these values, several evaluation metrics are computed, including the Dice coefficient, Intersection over Union (IoU), precision, recall, and specificity. These metrics measure how accurately the model segments the target abnormalities and provide a quantitative assessment of the segmentation performance.

3.5.3 Merlin

Merlin was used as a pretrained model without any additional training or fine-tuning. Unlike CT-CLIP and BiomedParse, Merlin does not rely on a predefined set of disease prompts for prediction.

Instead, the text descriptions provided with each ReXGroundingCT scan are used

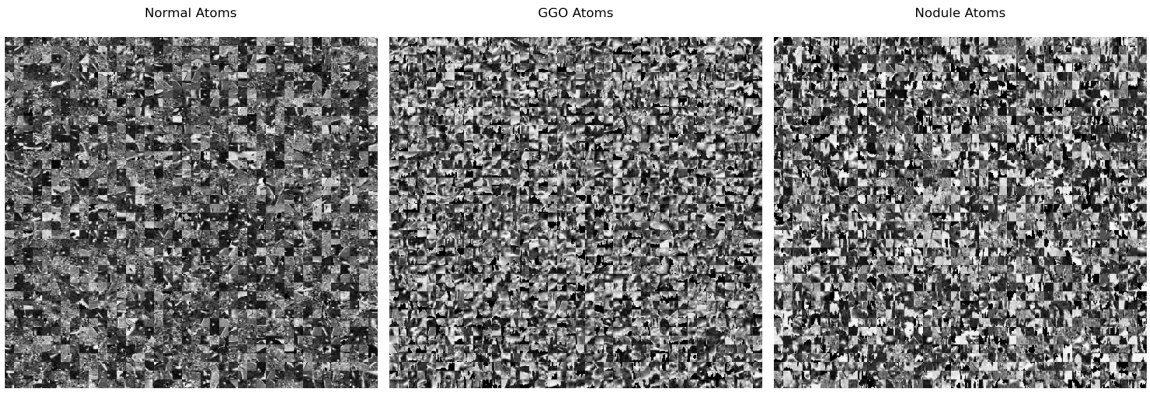

Figure 3.15: Randomly sampled dictionary atoms (centre slice of each $12 \times 12 \times 12$ atom) from the normal, GGO and nodule blocks of the trained dictionary.

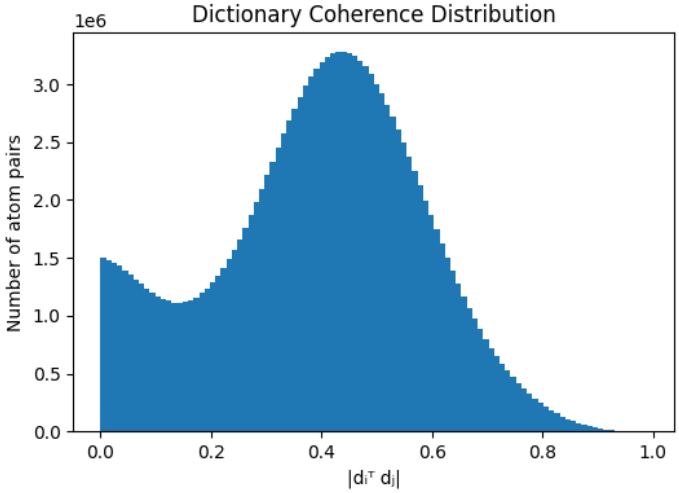

Figure 3.16: Distribution of pairwise mutual coherence $|d_i^\top d_j|$ across all dictionary atom pairs.

directly as input. These descriptions are converted into text embeddings, while the CT volume is processed to extract image features. The similarity between the text and image features is then calculated to identify the regions in the CT scan that correspond to the described finding.

Therefore, Merlin uses a zero-shot text-image matching approach to locate abnormalities. It does not use a separate classification head or segmentation decoder. Instead, the model identifies relevant regions based on the relationship between the CT image features and the provided text descriptions.

Preprocessing Each CT volume is loaded from a NIfTI file. Any missing or invalid (`NaN`) values are replaced with zero. If the input volume contains an additional channel dimension, making it a 4D volume instead of a 3D volume, the extra dimension is removed by averaging across the channel dimension.

The Hounsfield Unit (HU) values are then clipped to the range $[-1000, 400]$ and normalized to approximately the range $[0, 1]$ by adding 1000 and dividing by 1400. The resulting volume is converted into a single-channel 3D tensor and provided as input to the model.

This preprocessing uses a different intensity normalization strategy compared with the lung-segmentation-based normalization used in the sparse-coding pipeline described earlier in this chapter.

Inference The official pretrained Merlin model is loaded once, and a forward hook is attached to the last block of its 3D image encoder (`layer4` of the Inflated-3D ResNet). This allows the model to capture both the normal output and the detailed spatial features from the last encoder layer for each CT volume.

For each volume, the image encoder is executed once to obtain two outputs: a global image embedding representing the entire scan and a dense feature grid containing local spatial features. The global embedding is saved directly, while the dense feature grid is passed through Merlin's contrastive projection head and L2-normalised for the localisation process.

Each finding text associated with the scan is kept in its original form and separately encoded using Merlin's text encoder. The generated text embeddings are also L2-normalised. A voxel-wise cosine similarity is then computed between each finding-text embedding and every spatial location in the dense image feature grid. This produces one similarity map for each finding at the encoder's original downsampled resolution.

The similarity maps are resized back to the original CT volume resolution using trilinear interpolation and normalised to the range $[0, 1]$. These maps represent soft localisation regions showing how strongly each area matches the given finding text. They are not segmentation masks because the Merlin model was trained for image-text similarity rather than boundary prediction.

For reporting and comparison, each free-text finding is assigned to one of the four target categories using keyword matching. For example, mentions of "nodule" or "opacity" are mapped to their corresponding categories. This categorisation is only used for grouping localisation results and does not affect the model inference.

For every case, the generated finding-level localisation maps and the global image embedding are stored as compressed arrays. A JSON record is also saved containing the original finding text, the assigned category, the ground-truth category label, the ground-truth pixel count, and the norm of the image embedding. Previously processed cases are skipped to allow the pipeline to resume, and any failed cases are recorded in a separate error file and a run-level error list without interrupting the remaining processing.

3.5.4 BiomedParse

BiomedParse was used as a pretrained model without any additional training. The official released v2 checkpoint was applied directly to local CT volumes. Text prompts were used to guide the model to segment the same target abnormalities.

Preprocessing Each CT volume was loaded from a NIfTI file and rearranged into a depth-first (D, H, W) format. A standard lung window (width 1500 and level -160) was applied, and the intensity values were converted into an 8-bit image range of $[0, 255]$. This preprocessing method is different from the lung-based windowing used earlier in the sparse-coding pipeline. The preprocessing was performed once for each

case because only the text prompt changes when detecting different abnormalities in the same volume.

The processed volume was then passed through BiomedParse's `process_input` function. This function pads and resizes each axial slice to a fixed size of 512×512. The padding information and the axis where padding was applied were stored so that the predicted masks could later be converted back to the original CT volume size.

Inference The official pretrained BiomedParse v2 checkpoint was loaded using the model configuration provided by BiomedParse. The checkpoint was loaded from a local path if available; otherwise, it was downloaded from the model repository.

For each of the four target abnormalities (Lung nodule, Lung opacity, Consolidation, and Atelectasis), a text prompt was created by combining several disease-related descriptions using BiomedParse's `[SEP]` separator. For example, a lung nodule can be described as "lung nodule", "pulmonary nodule", or "nodular lesion". Combining these descriptions allows the model to consider different ways of describing the same abnormality in a single inference step.

The preprocessed CT volume and the generated text prompt were given to the model in evaluation mode. The model produced a predicted mask for each text description along with an object existence score indicating the confidence that the abnormality exists.

The predicted masks were resized back to 512×512 resolution and processed using BiomedParse's own post-processing method with a fixed probability threshold. The masks from all related text descriptions were then combined into one final mask for each abnormality category. The merged mask was converted into a binary foreground mask and transformed back to the original CT volume coordinates using the stored padding information.

Both a binary segmentation mask and a continuous probability map were saved. The probability map was stored separately so that different threshold values could be tested later without running the model again.

An abnormality was considered present only when both conditions were satisfied: the object existence score was above a predefined threshold and the predicted mask contained more than a minimum number of foreground voxels. This prevents incorrect detections caused by using only the confidence score or only the predicted mask size.

For each CT case, the generated binary masks, probability maps, visual overlays, and a JSON report were saved. The report contains the object existence score, mask size, final presence decision, and the thresholds used during inference. The report is first written to a temporary file and then safely replaced to prevent corruption if the process is interrupted. Cases that already have saved results are automatically skipped, allowing the complete pipeline to continue from where it stopped.

3.5.5 CT-CLIP

CT-CLIP was used as a pretrained model without any additional training. The released CT-LiPro linear-probe checkpoint was applied directly to the available CT volumes to predict the probability of different abnormalities.

Preprocessing Unlike the sparse-coding pipeline, which uses physical spacing information to resample CT volumes, CT-CLIP requires every input volume to have

the same fixed size because the model was pretrained using a specific input resolution.

For each CT volume, the Hounsfield Unit (HU) values were clipped to the range $[-1000, 1000]$ and divided by 1000 to scale the values approximately between $[-1, 1]$. The volume axes were then changed from (X, Y, Z) order to (Z, X, Y) order. Finally, each volume was resized using trilinear interpolation to a fixed size of $240 \times 480 \times 480$ before being given to the model.

Inference The released `CT_LiPro_v2.pt` checkpoint was loaded into the pretrained CT-CLIP model together with its linear classification head. The model weights were loaded, and any missing or unexpected parameters were recorded to ensure that the loading process was correct.

For each CT volume, an empty text prompt was tokenised and passed to the model together with the preprocessed CT image using `return_latents=True`. In this process, only the image features generated by the CT encoder were used. These image features were passed through the linear classification head followed by a sigmoid function to produce independent probability values for all 18 CT-RATE abnormalities at the same time.

The inference was performed on all available CT volumes from the local CT-RATE subset used in this project. Each volume was processed separately with gradients disabled to reduce memory usage. The predicted probabilities for all 18 abnormalities were saved in a CSV file.

The four abnormalities that match this project's target classes (Atelectasis, Lung nodule, Lung opacity, and Consolidation) were also saved separately together with their ground-truth labels. These results were stored in the same scan order as the accession list, allowing CT-CLIP to be evaluated using the same classification metrics as other deep learning models. The evaluation was performed on all 2,465 available CT scans.

A second inference method was also tested using CT-CLIP's original zero-shot capability without the trained linear classification head. In this approach, each abnormality was represented using two text prompts: "abnormality is present" and "abnormality is not present". The model compared the similarity between the CT volume and both prompts, and a softmax function was used to calculate the probability of the abnormality being present.

This zero-shot approach was tested successfully on a small pilot dataset, but it was not used for the final full-dataset evaluation.

3.6 Evaluation Metrics

The evaluation of medical image analysis systems requires multiple metrics because different measures capture different aspects of model behaviour. In chest CT analysis, classification metrics describe the ability of the model to identify abnormal findings, segmentation metrics measure spatial agreement with reference annotations, and interpretability metrics evaluate whether the learned representations provide meaningful explanations.

3.6.1 ReXRank

The ReXrank Leaderboard is a standardized public benchmarking platform for evaluating AI-driven chest X-ray report generation and visual question answering (VQA). It uses 8 distinct clinical and text metrics, relying on the composite score 1/RadCliQ-v1 as its primary ranking mechanism to measure real-world clinical alignment. Evaluation Framework and DataReXGradient Dataset: The core private test set comprises 10,000 multi-site studies across 67 institutions to test zero-shot generalization.Public Datasets: Incorporates standard benchmarks including MIMIC-CXR, IU-Xray, and CheXpert Plus.Task Splits: Separates models into findings-only and the more difficult findings + impression generation configurations.

3.6.2 Classification

For a binary classification problem, let TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively. Accuracy measures the overall proportion of correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{3.7}$$

Although accuracy provides an overall performance estimate, it can become misleading when the dataset contains class imbalance. Therefore, additional metrics based on positive class prediction are considered.

Precision indicates the reliability of positive predictions by measuring the fraction of detected abnormalities that are correct:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{3.8}$$

Recall, also referred to as sensitivity, measures the ability of the model to recover existing abnormal cases:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{3.9}$$

The F1-score combines precision and recall into a single measure and provides a balanced evaluation when both false positive and false negative errors are important:

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{3.10}$$

The Area Under the Receiver Operating Characteristic Curve (AUROC) evaluates the ability of a classifier to separate positive and negative samples over all possible decision thresholds. It is calculated from the relationship between the true positive rate (TPR) and false positive rate (FPR):

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR})\, d\text{FPR}. \tag{3.11}$$

A higher AUROC indicates that the model can better distinguish between abnormal and normal cases without depending on a particular classification threshold.

Average Precision (AP) summarizes the precision-recall relationship and is particularly useful for medical datasets where abnormal cases may represent a smaller proportion of the data:

$$\text{AP} = \sum_{n} (R_n - R_{n-1}) P_n, \tag{3.12}$$

where P_n and R_n represent precision and recall values at different classification thresholds. Higher average precision indicates better detection of positive cases under class imbalance.

The reliability of predicted probabilities is measured using Expected Calibration Error (ECE). The metric compares predicted confidence with the actual accuracy within confidence intervals:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|. \tag{3.13}$$

Lower ECE values indicate that the predicted confidence scores better reflect the actual likelihood of correctness.

3.6.3 Localization and Segmentation

For segmentation evaluation, let P represent the predicted region and G represent the ground-truth annotation. The Dice coefficient measures the overlap between the two regions:

$$\text{Dice}(P, G) = \frac{2|P \cap G|}{|P| + |G|}. \tag{3.14}$$

Intersection over Union (IoU), also known as the Jaccard index, measures the ratio between the intersection and union of the predicted and reference regions:

$$\text{IoU}(P, G) = \frac{|P \cap G|}{|P \cup G|}. \tag{3.15}$$

Dice and IoU provide complementary information about segmentation quality, where higher values indicate better spatial agreement between predicted and reference regions.

3.6.4 Interpretability of Sparse Dictionary Learning

In sparse dictionary learning, interpretability is obtained from the learned dictionary atoms and their corresponding sparse representations. Therefore, evaluation focuses on whether the dictionary provides compact, meaningful, and discriminative representations of CT image patterns.

The reconstruction error measures how effectively the learned dictionary represents the input data. Given an input patch x, dictionary D, and sparse coefficient vector α, the reconstruction error is defined as:

$$\text{Reconstruction Error} = \|x - D\alpha\|_2^2. \tag{3.16}$$

A lower reconstruction error indicates that the dictionary captures important structures present in the image data.

The sparsity of the representation describes how many atoms contribute to the reconstruction. It is measured using the number of non-zero coefficients:

$$\text{Sparsity} = \|\alpha\|_0. \tag{3.17}$$

Lower values indicate that only a small number of atoms are required to explain a sample, making the representation easier to interpret.

Dictionary coherence evaluates the similarity between different atoms in the learned dictionary. The mutual coherence is defined as:

$$\mu(D) = \max_{i \neq j} \frac{|d_i^T d_j|}{\|d_i\|_2 \|d_j\|_2}. \tag{3.18}$$

A lower coherence value indicates that the dictionary contains less redundant atoms and that individual atoms capture more distinct image characteristics.

Atom utilization measures how frequently each dictionary atom contributes to the representation of the dataset:

$$U_k = \sum_{i=1}^{N} \mathbb{1}(\alpha_{k,i} \neq 0), \tag{3.19}$$

where U_k represents the activation frequency of atom k. This metric helps identify whether the learned dictionary is effectively utilized or dominated by a limited number of atoms.

For supervised dictionary learning approaches such as LC-KSVD, class-specific atom activation provides additional interpretability. Disease-related atoms are expected to activate more frequently for samples belonging to their associated class compared with unrelated classes. This indicates that the learned atoms capture discriminative patterns rather than only general image structures.

Finally, visual inspection of reconstructed dictionary atoms is used as a qualitative interpretability assessment. By examining representative atoms, their relationship with anatomical structures or pathological appearances can be analysed, providing an intuitive understanding of the features learned by the model.

Chapter 4

Results

4.1 Preprocessing Validation

4.1.1 Lung Segmentation Quality (TotalSegmentator vs Ground Truth)

4.2 Classical Image Processing Baseline

4.3 Sparse-Coding Dictionary Learning (K-SVD Family)

4.3.1 K-SVD (Unsupervised)

4.3.2 FDDL (Fisher Discrimination Dictionary Learning)

4.3.3 Frozen Dictionary Learning

Sparse Dictionary Interpretability

Sparsity and Atom Utilisation

Atom Co-occurrence and Clustering

Dictionary Coherence

Visual Inspection of Atoms and Reconstructions

Chapter 5

Conclusion

Appendix A

Essential English Grammar - A Handout

The Appendices are numbered as A, B,.. and so on. Sections and Subsections of the appendix then would be A.1, A.1.1, A.2 etc.

A.1 Simple Present Tense

A.2 Perfect Tense

A.3 Past Past Perfect Tense

A.4 Passive Voice

A.5 Technical Writing Fundamentals

A.6 Proof Reading

Appendix B

Graphical Interpretation of Data

The graphical interpretation given below shows the relationship between x and y is acceptable to the purpose of the project

B.1 Data Analysis

B.1.1 Statistical Analysis

B.1.2 Empirical Analysis

B.2 The X vs Y vs Z

Bibliography

[1] C. J. Murray, "The global burden of disease study at 30 years," *Nature medicine*, vol. 28, no. 10, pp. 2019–2026, 2022.

[2] K. Doi, "Computer-aided diagnosis in medical imaging: historical review, current status and future potential," *Computerized medical imaging and graphics*, vol. 31, no. 4-5, pp. 198–211, 2007.

[3] A. Borakati, A. Perera, J. Johnson, and T. Sood, "Diagnostic accuracy of x-ray versus ct in covid-19: a propensity-matched database study," *BMJ open*, vol. 10, no. 11, p. e042946, 2020.

[4] G. McCreadie and T. Oliver, "Eight ct lessons that we learned the hard way: an analysis of current patterns of radiological error and discrepancy with particular emphasis on ct," *Clinical radiology*, vol. 64, no. 5, pp. 491–499, 2009.

[5] M. Ennab and H. Mcheick, "Advancing ai interpretability in medical imaging: a comparative analysis of pixel-level interpretability and grad-cam models," *Machine Learning and Knowledge Extraction*, vol. 7, no. 1, p. 12, 2025.

[6] N. L. S. T. R. Team, "The national lung screening trial: overview and study design," *Radiology*, vol. 258, no. 1, pp. 243–253, 2011.

[7] H. J. de Koning, C. M. van Der Aalst, P. A. de Jong, E. T. Scholten, K. Nackaerts, M. A. Heuvelmans, J.-W. J. Lammers, C. Weenink, U. Yousaf-Khan, N. Horeweg, *et al.*, "Reduced lung-cancer mortality with volume ct screening in a randomized trial," *New England journal of medicine*, vol. 382, no. 6, pp. 503–513, 2020.

[8] P. J. Withers, C. Bouman, S. Carmignato, V. Cnudde, D. Grimaldi, C. K. Hagen, E. Maire, M. Manley, A. Du Plessis, and S. R. Stock, "X-ray computed tomography," *Nature Reviews Methods Primers*, vol. 1, no. 1, p. 18, 2021.

[9] K. Doi, "Computer-aided diagnosis in medical imaging: Historical review, current status and future potential," *Computerized Medical Imaging and Graphics*, vol. 31, no. 4, pp. 198–211, 2007. Computer-aided Diagnosis (CAD) and Image-guided Decision Support.

[10] H.-P. Chan, K. Doi, C. Vyborny, K. Lam, and R. Schmidt, "Computer-aided detection of microcalcifications in mammograms. methodology and preliminary clinical study," *Investigative radiology*, vol. 23, pp. 664–71, 10 1988.

[11] T. W. Freer and M. J. Ulissey, "Screening mammography with computer-aided detection: Prospective study of 12,860 patients in a community breast center," *Radiology*, vol. 220, no. 3, pp. 781–786, 2001. PMID: 11526282.

[12] F. Li, M. Aoyama, J. Shiraishi, H. Abe, Q. Li, K. Suzuki, R. Engelmann, S. Sone, H. Macmahon, and K. Doi, "Radiologists' performance for differentiating benign from malignant lung nodules on high-resolution ct using computer-estimated likelihood of malignancy," *AJR. American journal of roentgenology*, vol. 183, pp. 1209–15, 12 2004.

[13] J. Shiraishi, H. Abe, R. Engelmann, M. Aoyama, H. MacMahon, and K. Doi, "Computer-aided diagnosis to distinguish benign from malignant solitary pulmonary nodules on radiographs: Roc analysis of radiologists' performance—initial experience," *Radiology*, vol. 227, no. 2, pp. 469–474, 2003. PMID: 12732700.

[14] K. Doi, "Computer-aided diagnosis in medical imaging: Historical review, current status and future potential," *Computerized Medical Imaging and Graphics*, vol. 31, no. 4, pp. 198–211, 2007. Computer-aided Diagnosis (CAD) and Image-guided Decision Support.

[15] T. Zhao, Y. Gu, J. Yang, N. Usuyama, H. H. Lee, S. Kiblawi, T. Naumann, J. Gao, A. Crabtree, J. Abel, *et al.*, "A foundation model for joint segmentation, detection and recognition of biomedical objects across nine modalities," *Nature methods*, vol. 22, no. 1, pp. 166–176, 2025.

[16] J. Wasserthal, H. C. Breit, M. T. Meyer, *et al.*, "Totalsegmentator: Robust segmentation of 104 anatomic structures in ct images," *Radiology: Artificial Intelligence*, vol. 5, no. 5, p. e230024, 2023.

[17] Z. Yang, Z. Chen, Y. Sun, A. Strittmatter, A. Raj, A. Allababidi, J. S. Rink, and F. G. Zöllner, "seg2med: a segmentation-based medical image generation framework using denoising diffusion probabilistic models," *arXiv e-prints*, pp. arXiv–2504, 2025.

[18] L. Blankemeier, A. Kumar, J. P. Cohen, J. Liu, L. Liu, D. Van Veen, S. J. S. Gardezi, H. Yu, M. Paschali, Z. Chen, J.-B. Delbrouck, E. Reis, R. Holland, C. Truyts, C. Bluethgen, Y. Wu, L. Lian, M. E. K. Jensen, S. Ostmeier, M. Varma, J. M. J. Valanarasu, Z. Fang, Z. Huo, Z. Nabulsi, D. Ardila, W.-H. Weng, E. A. Junior, N. Ahuja, J. Fries, N. H. Shah, G. Zaharchuk, M. Willis, A. Yala, A. Johnston, R. D. Boutin, A. Wentland, C. P. Langlotz, J. Hom, S. Gatidis, and A. S. Chaudhari, "Merlin: a computed tomography vision–language foundation model and dataset," *Nature*, vol. 652, p. 1318–1328, Mar. 2026.

[19] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang, "Segment anything in medical images," *Nature Communications*, vol. 15, p. 654, 2024.

[20] N. Ravi, J. Ma, Y. He, B. Wang, *et al.*, "Medsam2: Segment anything in 3d medical images and videos," *arXiv preprint arXiv:2504.03600*, 2025.

[21] E. M. Ayllon *et al.*, "Can foundation models really segment tumors? a benchmarking odyssey in lung ct imaging," *arXiv preprint arXiv:2505.01239*, 2025.

[22] Meta AI, "Sam 3: Segment anything with concepts." Submitted to ICLR 2026 (under review), 2025.

[23] A. Liu, R. Xue, X. R. Cao, Y. Shen, Y. Lu, X. Li, Q. Chen, and J. Chen, "Medsam3: Delving into segment anything with medical concepts," *arXiv preprint arXiv:2511.19046*, 2025.

[24] I. E. Hamamci, S. Er, *et al.*, "Ct-clip: Contrastive vision-language pretraining for chest ct," *arXiv preprint arXiv:2403.17834*, 2024.

[25] I. E. Hamamci, S. Er, E. Almas, E. Keles, E. Keles, M. Basar, A. Gungor, and G. Unal, "Ct-rate: A large-scale dataset for chest ct radiology report generation and abnormality detection," *arXiv preprint arXiv:2401.07515*, 2024.

[26] E. Tjoa and C. Guan, "A survey on explainable artificial intelligence (xai): Toward medical xai," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 11, pp. 4793–4813, 2021.

[27] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, ACM, 2016.

[28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[29] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[30] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks," in *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 839–847, IEEE, 2018.

[31] H. Wang, Z. Wang, M. Du, F. Yang, Z. Zhang, S. Ding, P. Mardziel, and X. Hu, "Score-cam: Score-weighted visual explanations for convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 111–119, 2020.

[32] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "Layercam: Exploring hierarchical class activation maps for localization," *IEEE transactions on image processing*, vol. 30, pp. 5875–5888, 2021.

[33] V. Hassija, V. Chamola, A. Mahapatra, A. Singal, D. Goel, K. Huang, S. Scardapane, I. Spinelli, M. Mahmud, and A. Hussain, "Interpreting black-box models: a review on explainable artificial intelligence," *Cognitive Computation*, vol. 16, no. 1, pp. 45–74, 2024.

[34] L. Ribeiro-Flucht, X. Chen, and D. Meurers, "Explainable ai in language learning: Linking empirical evidence and theoretical concepts in proficiency and readability modeling of portuguese," in *Proceedings of the 19th workshop on innovative use of NLP for building educational applications (BEA 2024)*, pp. 199–209, 2024.

[35] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.

[36] R. Guidotti, "Counterfactual explanations and how to find them: literature review and benchmarking: R. guidotti," *Data Mining and Knowledge Discovery*, vol. 38, no. 5, pp. 2770–2824, 2024.

[37] M. Van Lent, W. Fisher, and M. Mancuso, "An explainable artificial intelligence system for small-unit tactical behavior," in *Proceedings of the national conference on artificial intelligence*, pp. 900–907, Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, 2004.

[38] H. Zheng, H. Yong, and L. Zhang, "Deep convolutional dictionary learning for image denoising," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 630–641, 2021.

[39] Y. Li, S. Ding, and Z. Li, "Dictionary learning with the cosparse analysis model based on summation of blocked determinants as the sparseness measure," *Digital Signal Processing*, vol. 48, pp. 298–309, 2016.

[40] H. Lee, A. Battle, R. Raina, and A. Ng, "Efficient sparse coding algorithms," in *Advances in Neural Information Processing Systems* (B. Schölkopf, J. Platt, and T. Hoffman, eds.), vol. 19, MIT Press, 2006.

[41] M. Aharon, M. Elad, and A. Bruckstein, "K-svd: An algorithm for designing overcomplete dictionaries for sparse representation," *IEEE Transactions on Signal Processing*, vol. 54, no. 11, pp. 4311–4322, 2006.

[42] A. Yahyaoui, N. Fnaiech, and F. Fnaiech, "A suitable initialization procedure for speeding a neural network job-shop scheduling," *IEEE Transactions on Industrial Electronics*, vol. 58, no. 3, pp. 1052–1060, 2011.

[43] Y. Pati, R. Rezaiifar, and P. Krishnaprasad, "Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition," in *Proceedings of 27th Asilomar Conference on Signals, Systems and Computers*, pp. 40–44 vol.1, 1993.

[44] M. Aharon, M. Elad, and A. Bruckstein, "K-svd: An algorithm for designing overcomplete dictionaries for sparse representation," *IEEE Transactions on Signal Processing*, vol. 54, no. 11, pp. 4311–4322, 2006.

[45] Z. Jiang, Z. Lin, and L. S. Davis, "Label consistent k-svd: Learning a discriminative dictionary for recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 11, pp. 2651–2664, 2013.

[46] M. Yang, L. Zhang, X. Feng, and D. Zhang, "Fisher discrimination dictionary learning for sparse representation," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 543–550, 2011.

[47] B. T. Carroll, B. M. Whitaker, W. Dayley, and D. V. Anderson, "Outlier learning via augmented frozen dictionaries," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 6, pp. 1207–1219, 2017.

[48] T. Tong, J. Caballero, K. Bhatia, and D. Rueckert, "Dictionary learning for medical image denoising, reconstruction, and segmentation," in *Machine learning and medical imaging*, pp. 153–181, Elsevier, 2016.

[49] M. Elad and M. Aharon, "Image denoising via sparse and redundant representations over learned dictionaries," *IEEE Transactions on Image processing*, vol. 15, no. 12, pp. 3736–3745, 2006.

[50] L. Bao, M. Robini, W. Liu, and Y. Zhu, "Structure-adaptive sparse denoising for diffusion-tensor mri," *Medical image analysis*, vol. 17, no. 4, pp. 442–457, 2013.

[51] L. Shi, Y. Chen, H. Shu, L. Luo, C. Toumoulin, and J.-L. Coatrieux, "Low-dose ct image processing using artifact suppressed dictionary learning," in *2014 IEEE 11th International Symposium on Biomedical Imaging (ISBI)*, pp. 1127–1130, IEEE, 2014.

[52] S. Ravishankar and Y. Bresler, "Mr image reconstruction from highly undersampled k-space data by dictionary learning," *IEEE Transactions on Medical Imaging*, vol. 30, no. 5, pp. 1028–1041, 2011.

[53] A. Rueda, N. Malpica, and E. Romero, "Single-image super-resolution of brain mr images using overcomplete dictionaries," *Medical image analysis*, vol. 17, no. 1, pp. 113–132, 2013.

[54] M. Liu, L. Lu, X. Ye, S. Yu, and M. Salganicoff, "Sparse classification for computer aided diagnosis using learned dictionaries," in *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 41–48, Springer, 2011.

[55] P. Wu, K. Xia, Y. Zhang, X. Qian, G. Wang, and H. Yu, "Dictionary learning-based ct detection of pulmonary nodules," in *Developments in X-Ray Tomography X*, vol. 9967, pp. 88–99, SPIE, 2016.

[56] S. Kong and D. Wang, "A dictionary learning approach for classification: Separating the particularity and the commonality," in *European conference on computer vision*, pp. 186–199, Springer, 2012.

[57] A. Ben-Cohen, E. Klang, A. Kerpel, E. Konen, M. M. Amitai, and H. Greenspan, "Fully convolutional network and sparsity-based dictionary learning for liver lesion detection in ct examinations," *Neurocomputing*, vol. 275, pp. 1585–1594, 2018.

[58] P. Rajpurkar *et al.*, "Rexgroundingct: A large-scale dataset for grounded radiology report generation on chest ct," *arXiv preprint*, 2024.

[59] X. Zhang, H.-Y. Zhou, X. Yang, O. Banerjee, J. N. Acosta, J. Miller, O. Huang, and P. Rajpurkar, "Rexrank: A public leaderboard for ai-powered radiology report generation," 2024.

[60] J.-x. Yang, M. Zhang, Z.-h. Liu, L. Ba, J.-x. Gan, and S.-w. Xu, "Detection of lung atelectasis/consolidation by ultrasound in multiple trauma patients with mechanical ventilation," *Critical Ultrasound Journal*, vol. 1, no. 1, pp. 13–16, 2009.

[61] Z. Zhao, L. Dai, Y. Zhang, W. Xie, and Y. Wang, "Rethinking whole-body ct image interpretation: An abnormality-centric approach," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5179–5189, 2026.

[62] R. Rubinstein, M. Zibulevsky, and M. Elad, "Efficient implementation of the k-svd algorithm using batch orthogonal matching pursuit," Tech. Rep. CS-2008-08, Technion - Israel Institute of Technology, 2008.